OOP with C++ (2BTCO303) — Previous Year Question Papers with Model Answers

Dr. Subhash University · B.Tech CSE (AI/ML) · Semester 3 · Winter 2024 + Winter 2025 Source folder analysed: `c++` (Google Drive) — 11 PDFs → **2 actual exam papers**, 7 unit-wise notes. (Two files named "Fast Learner List" are class merit lists, not question papers; they are excluded.)

Exam format (identical both years): **5 questions, all compulsory, internal OR choice in Q2–Q5, 100 marks, 3 hours.** Q.1 = 10 × 2-mark short answers **[20]** · Q.2 = 4+6+10 **[20]** · Q.3 = 4+6+10 **[20]** · Q.4 = 4+6+10 **[20]** · Q.5 = 4+6+10 **[20]**

Syllabus units: **U1** Concepts of OOP · **U2** C++ Basics · **U3** Functions · **U4** Objects and Classes · **U5** Polymorphism · **U6** I/O and File Management · **U7** Template, Exceptions and STL.

> **Every program in this answer key was compiled and run** with clang++ 23.1.4 (`-std=c++17`) before being written here. The "Output:" blocks are the program's actual printed output, not a prediction.

---

1. What actually repeats (the paper-setter's fingerprints)

Topic	W2024	W2025	Unit	Marks	Verdict
Friend function with example	Q2(C)	Q4(C)	3/4	**10**	10 marks in **both** years. Highest-probability long answer.
C++ program structure (sections)	Q2(A)	Q5(C)	2	4 → **10**	Both years; grew from 4 to 10 marks. Learn every section name.
Templates: class + function	Q5(B) class template, Q1(J)	Q2(C)	7	6 → **10**	Both years, escalating marks.
Exception handling	Q5(C) user-defined + division-by-zero	Q5(A) try/throw	7	**10** + 4	Both years; a program is always asked.
Function overloading	Q3(C)	Q3(B) + Q5(C) OR (volume program)	3/5	**10**	Both years; 2025 wants a *working program with output*.
Function overriding	Q3(C) OR	Q3(B) differentiate	5	**10** / 6	Both years.
Constructor & destructor	Q1(G), Q3(B) default ctor	Q4(A) OR, Q5(B) OR program	4	2–10	Both years.
setw / setprecision / manipulators	Q5(B) OR	Q4(B) OR	6	**6**	Both years, same slot.
Stream class hierarchy diagram	Q3(A) OR	Q4(B)	6	4–6	Both years.
Inheritance types + diagrams	Q1(H) multiple, Q4(C) OR multilevel	Q2(A) single + multilevel, Q4(C) OR diamond	4/5	4–10	Both years — a diagram is guaranteed.
Class and object	Q1(B) + Q3(A) program	Q1(B)	1/4	2–4	Both years.
Variable vs Constant	Q1(A)	Q1(F)	2	2	Both years, Q.1 slot A/F.
Operators (list / unary vs binary)	Q1(E)	Q1(G), Q3(B) OR	2/6	2–6	Both years.
Data types	Q1(D)	Q1(I)	2	2	Both years.
Encapsulation / abstraction / inheritance concepts	Q1(C), Q1(I)	Q1(C), Q1(D), Q3(A) OR	1	2–4	Both years.
`using namespace std;` / link section	Q2(A) implied	Q1(E), Q1(J), Q5(A) OR macro	2/4	2–4	New emphasis in 2025.
Virtual function (runtime polymorphism)	Q2(C) OR	Q4(A) late binding	5	10 / 4	Strong candidate.
Static data member	Q3(B) OR	—	4	6	Expected to return.
Operator overloading	Q4(C)	—	5	10	Expected to return.
Call by value (vs reference)	Q2(B)	—	3	6	Strong candidate — the mirror question (call by reference) is unwritten so far.

Topic	W2024	W2025	Unit	Marks	Verdict
Scope resolution operator `::`	Q4(A)	—	2/4	4	Frequently recycled.
Default arguments	—	Q2(C) OR	3	**10**	New in 2025.
Typecasting: implicit vs explicit	—	Q3(C)	2	**10**	New in 2025.
Control structures + if/if-else/switch	Q1(F) while loop	Q3(C) OR	2	**10**	New in 2025.
Pointers	Q1(H) dot/star implied	Q5(B), Q1(H)	2	6	New in 2025 as a long-ish answer.
public / private / protected in derivation	—	Q2(B)	4	6	New in 2025.
STL in detail	Q4(B) OR	—	7	**6**	Untested in 2025 → likely return.
File modes / binary files	Q4(A) OR, Q5(A) OR	—	6	4	Untested in 2025 → likely return.
Multiple inheritance (definition + diagram)	Q1(H)	Q4(C) OR diamond	1/4	2–10	2025's diamond question is the advanced form of 2024's.

Where the marks sit. Three topics alone — **Friend function, C++ program structure, and Templates** — carry 10 marks in both papers, and **exception handling with a program** carries another 10. That is roughly **30 of 100 marks** from three answers. Add overloading/overriding and the constructor–destructor program and you have 50.

The clearest trend: 2025 moved from theory toward working code. Winter 2024 asks "explain X with example" (a snippet is enough); Winter 2025 repeatedly says *"Construct a program… Present the output of the program"* and *"write a program to demonstrate…"*. So for 2026, a runnable program with its **printed output pasted underneath** is the answer shape that earns full marks. Every program in §3–§4 below is already executed for you.

Second: the setter rotates, it does not repeat verbatim the way the DBMS paper does. Only *Friend function*, *program structure*, *templates* and *manipulators* recur in the same wording — everything else comes back in a different guise (multiple inheritance → diamond problem; while loop → control structures; `setw` → `<iomanip>` and `<IOMANIP>`).

---

2. The 20-mark Q.1 bank — short answers (both years, all 20 items)

Fastest marks in the paper: 10 questions × 2 marks, pure recall. Two to five lines each, ideally with one line of code.

Winter 2024 — Q.1

(A) Define variable and constant. A **variable** is a named memory location whose value can change during program execution; it must be declared with a data type before use: `int age = 21; age = 22;` (legal). A **constant** is a named memory location whose value is fixed for the whole program and cannot be modified after initialisation: `const float PI = 3.14;` (reassigning it is a compile-time error). Constants may also be created with a macro `#define PI 3.14` or, since C++11, `constexpr`. Differences: a `const` variable is type-checked and obeys scope; `#define` is pure textual substitution by the preprocessor and has no type.

(B) Define class and object. A **class** is a user-defined blueprint/template that binds data (data members) and the functions that operate on it (member functions) into one unit, with access specifiers controlling visibility; it is a logical entity that occupies no memory until instantiated. An **object** is a real instance of a class — a physical entity that occupies memory; the class is declared once, many objects are created.

```
class Student { int roll; public: void show(); };  // class
Student s1, s2;                                    // two objects
```

(C) List the basic concepts of OOP. 1. **Class** — blueprint of data and functions. 2. **Object** — instance of a class. 3. **Data Abstraction** — showing only necessary features, hiding implementation. 4. **Encapsulation** — wrapping data and code together and restricting direct access (data hiding). 5. **Inheritance** — deriving a new class from an existing one, enabling reuse. 6. **Polymorphism** — one interface, many forms (compile-time overloading, run-time virtual functions). 7. **Dynamic memory allocation** — creating objects at run time with `new` / `delete`. (Add *message passing* if your notes list it: objects communicate by calling each other's member functions.)

(D) Explain data types. A data type decides the size of a variable, the range of values it can hold, and the operations permitted on it. C++ has three families: * **Built-in / fundamental** — `char` (1 byte), `int` (4), `float` (4, ~6 digits), `double` (8, ~15), `long double` (8/16), `bool` (1), `void`, plus modifiers `signed`, `unsigned`, `short`, `long`. * **Derived** — array, function, **pointer**, reference. * **User-defined** — `class`, `struct`, `union`, `enum`, `typedef`. Type sizes are implementation-dependent; `sizeof(type)` reports them at compile time.

(E) List out operators. Arithmetic `+ - * / %` · Relational `< > <= >= == !=` · Logical `&& || !` · Assignment `= += -= *= /= %=` · **Increment/decrement** `++ --` · Bitwise `& | ^ ~ << >>` · Ternary/conditional `? :` · **Scope resolution** `::`, member access `.` and pointer-member `->`, `.*` and `->*` · Comma `,` · `sizeof` · Casting `static_cast` / `reinterpret_cast` / `const_cast` / `dynamic_cast` · `new` / `delete`. By arity they divide into **unary** (`-a`, `!a`, `++a`, `*p`, `&x`), **binary** (`a+b`, `a%b`) and the single **ternary** operator.

(F) Explain while loop. A **pre-tested (entry-controlled) loop** used when the number of iterations is not known in advance. The condition is evaluated **before** each iteration, so the body may run **zero times**.

```
int n = 5;
while (n > 0) { cout << n << " "; --n; }     // 5 4 3 2 1
```

Flow: test condition → if true execute body → go back to test → if false exit. The counter/accumulator variable must be initialised before the loop and updated inside it, otherwise the loop never terminates (infinite loop). `do-while` is the post-tested variant that always runs at least once; `break` exits early, `continue` skips to the next test.

(G) Define constructor and destructor. A **constructor** is a special member function with the **same name as the class**, **no return type** (not even void), invoked **automatically** when an object is created, used to initialise the object's data members. It may be overloaded, may take default arguments, and is usually public. A **destructor** is a special member function named `~ClassName`, with no arguments and **no return type**, invoked **automatically when the object goes out of scope or is deleted**, used to release resources (memory, open files, connections). A class has **one** destructor only — it cannot be overloaded — and if none is written the compiler supplies a default one.

```
class File {
  public: File()  { cout << "built\n"; }     // constructor
          ~File() { cout << "released\n"; }  // destructor
};
```

(H) Design a diagram of multiple inheritance. One derived class inherits from **two or more base classes** at the same level.

```
  [ Base1 ]        [ Base2 ]
    roll,name        marks
       \               /
        \             /          <- both arrows point down
         [  Derived  ]
           display()
```

```
class Base1 { protected: int roll; string name; };
class Base2 { protected: int marks; };
class Derived : public Base1, public Base2 { public: void display(); };
```

The derived class gets all non-private members of both bases; a name present in both bases is **ambiguous** and must be qualified — `Derived::Base1::show()` — which is exactly the problem *virtual* inheritance solves (the diamond).

(I) Explain the benefits of OOP. * **Reusability** — inheritance and ready-made classes/STL avoid rewriting code. * **Modularity / simplicity** — a program is a set of independent objects, each small and manageable. * **Data hiding & security** — `private` members and accessors protect internal state. * **Maintainability / easy modification** — a change inside one class does not ripple through the program. * **Polymorphism / flexibility** — one interface serving many types; behaviour chosen at run time. * **Scalability** — an application grows by adding new classes instead of rewriting old logic. * **Real-world modelling** — objects mirror entities, so complex systems are easier to design. * **Reduced cost & better software quality**, message passing between objects, and libraries that survive project after project. (Contrast with procedural POP: no security of data, global data visible to every function, harder to maintain as programs grow.)

(J) Define template. A **template** is the C++ mechanism for **generic programming**: a class or function written once with a **placeholder type parameter**, from which the compiler generates a separate concrete version for each type actually used (**compile-time polymorphism**). It removes duplicated code for type-varying algorithms.

```
template <typename T>          // function template
T maximum(T a, T b) { return (a > b) ? a : b; }

template <class T, int N>      // class template
class Array { T arr[N]; /* ... */ };
```

Two kinds: **function template** and **class template**. Type can be explicit `maximum<int>(3,7)` or inferred `maximum(3,7)`.

Winter 2025 — Q.1

(A) What do the acronyms OOP and POP stand for? POP = Programming Oriented Programming — properly **Procedure-Oriented Programming**: the program is a sequence of functions/procedures operating on data, with the logic written first and data moving freely between functions (C, Pascal). **OOP = Object-Oriented Programming**: the program is organised around **objects** that bundle data with the functions acting on it, with the data protected first and functions designed around it (C++, Java). Key contrast: POP is top-down, has no data hiding, and reuses code through function calls; OOP is bottom-up, uses inheritance and access control, and reuses through classes and objects.

(B) Define a class in the context of C++. A class is a **user-defined data type** and the building block of OOP: a template that groups data members and member functions under one name with access specifiers (`private` by default, `public`, `protected`), providing abstraction and data hiding. Declared with the keyword `class`, terminated by a `;`. It is a logical blueprint only — no memory is allocated until objects are created. Members are reached with `.` from an object and `->` from a pointer; private data can be reached only by the class's own functions and declared `friend`s.

(C) Define encapsulation. Encapsulation is wrapping **data and the code that manipulates it into a single unit (the class)** while restricting direct outside access to the internal state — achieved with `private` members plus public `get`/`set` functions. It is the mechanism that delivers *data hiding*: implementation hidden, interface exposed.

```
class Account {
    double balance;                       // hidden
  public:
    void deposit(double a) { if (a > 0) balance += a; }
    double getBalance() const { return balance; }
};
```

Benefit: the invariant can be validated in one place, and the internal representation can change without breaking users. (Abstraction is the related *idea* of showing only essentials; encapsulation is the *implementation* of that idea.)

(D) Describe the key idea of inheritance and interpret its significance in OOP. Inheritance is the mechanism by which a new class (**derived/subclass**) acquires the properties — data members and member functions — of an existing class (**base/superclass**), written `class Derived : access-specifier Base;`. Its significance: it expresses **"is-a"** relationships directly in code, gives **reusability** without rewriting or copying tested base code, supports **hierarchical classification** and extensibility (add behaviour by adding subclasses), enables **runtime polymorphism** through virtual functions and base-class pointers, and reduces redundancy by putting shared behaviour once in the base. Cost to acknowledge: deep or multiple inheritance adds coupling and can create ambiguity (the diamond), which `virtual` inheritance resolves. Five forms: single, multiple, multilevel, hierarchical, hybrid.

(E) State the purpose of the statement `using namespace std;` in C++. It **imports the standard namespace** `std`, in which all standard library names live (`cout`, `cin`, `string`, `vector`), so they can be written unqualified. Without it every use needs the scope-resolution prefix: `std::cout << "hi";`. Purpose: save repetition and make code shorter. Rule of thumb worth stating: acceptable in a small program or inside `main`, but in headers and large projects it pollutes the global namespace and can collide with your own names — prefer explicit `std::` qualification. It is a *directive*, not a declaration; it introduces no symbols itself and has no run-time cost.

(F) Define the term constant and how is it different from a variable? A **constant** is a fixed value that cannot be altered after it is established. In C++ it is created with `const` (read-only variable, type-checked, must be initialised at declaration), `constexpr` (constant evaluated at compile time), an enumerator, or `#define` (a preprocessor macro, textual substitution with no type or scope). Difference from a variable: a **variable's** value may be assigned and reassigned any number of times during execution and it denotes a memory cell whose contents change; a **constant's** value is set once and any later attempt to modify it is a **compile-time error**. `const int MAX = 10; MAX = 11;` → error. Naming convention: constants are usually written in upper case.

(G) Identify unary and binary operator expressions: (I) `a++` and `--a` (II) `a+b` and `a-b`. (I) `a++` and `--a` are UNARY operators — each acts on a **single** operand. `++`/`--` are increment/decrement; `a++` is **postfix** (use the old value, then increment) and `++a` is **prefix** (increment first, then use). Other unary operators: `-a` (negation), `!a` (logical not), `~a` (bitwise not), `*p` (indirection), `&x` (address-of), `sizeof T`, `+a` (unary plus). **(II) `a+b` and `a-b` are BINARY operators** — each needs **two** operands (left and right). The whole arithmetic set `+ - * / %`, relational, logical, bitwise, assignment, comma, and the member operators `.` and `->` are binary. Extra note for full marks: C++ also has exactly one **ternary** operator, the conditional `?:`. Overloaded operators keep their arity except the new form `operator()`.

(H) Define the use of the dot operator (.) and the () operator.* **Dot (.) — class member access operator. Connects an **object** (or a reference) to one of its members: `s.roll = 10; s.display();`. Its left side must be an object, and only public members are reachable from outside the class. **Asterisk (*) has three distinct uses.**

(1) As a **declaration operator**, it makes a variable a **pointer**: `int *p = &a;`. (2) As the **indirection / dereference operator**, it reads or changes the value at the address held: `*p = 25;`. (3) As the **multiplication** binary operator: `a * b`. In inheritance, the pointer-to-member form `->*` complements `.*`. When accessing a member through a **pointer** to an object you use the arrow instead of the dot: `p->display()` is exactly `(*p).display()`.

(I) List any four basic data types in C++. `int` — integers, 4 bytes, e.g. `int roll = 101;` · `float` — single-precision real, 4 bytes, ~6 digits, `float price = 99.5f;` · `double` — double-precision real, 8 bytes, ~15 digits, `double pi = 3.141592653589;` · `char` — a single character, 1 byte, `char grade = 'A';`. (Fifth worth adding: `bool` — `true` / `false`, 1 byte.) Each has signed/unsigned/short/long modifiers, and `void` is the "no value" type.

(J) Recognize the purpose of the Link section in a C++ program structure. The **link section is the first executable section of a C++ program: it tells the compiler which library or header files must be linked** so their predefined functions and objects can be used. Written as

```
#include <iostream>        // header, brackets = standard library
#include "myheader.h"      // quotes  = user-defined header in the project
#define PI 3.14            // macro constants also belong to this section
using namespace std;
```

Without it `cout`, `cin`, `sqrt`, `string` are unknown symbols and compilation fails. `#include` is handled by the **preprocessor**, which pastes the header's contents into the source before compilation; the compiler then records the external references and the **linker** resolves them against the library binaries, producing the executable.

---

3. Winter Examination 2024 — full solved paper

Date 23/12/2024 · 3 hours · 100 marks

Q.2 (A) Draw and explain the C++ program structure. [4]

Nine sections, in this order. Write the numbered list, the diagram, and the small sample program — that is the full 4 marks.

```
1. Documentation / comment section   // name, date, purpose  (ignored by compiler)
2. Link section                      #include <iostream>   #define PI 3.14
3. Definition section                const int X = 5;   typedef / enum / using
4. Global declaration section        int count;          visible to all functions
5. Class declaration section         class Student { ... };
6. Function declaration section      void show();        prototypes
7. Main() function section           int main() { ... return 0; }
8. Sub-declaration section           local variables, first inside main
9. Sub-program definition section    void show() { ... }   function bodies
```

1. **Documentation section** — comments (`//`, `/* */`) describing author, purpose, date; improves readability, removed before compilation.
2. **Link section** — the first thing the compiler actually sees: `#include` directives that **link** header files of the standard or user library so their functions become available (`<iostream>` for `cin` / `cout`). Also holds `#define` macros. Angle brackets search the system include path; quotes search the current directory first.
3. **Definition section** — `const` and symbolic definitions, `typedef`, `enum`, `using` declarations that give names to values and types used throughout.

4. **Global declaration section** — variables/functions declared **outside every function and class**, hence visible to all of them and alive for the whole run. Discouraged because they break encapsulation, but standard in this structure.
5. **Class declaration section** — the data and function members of each user-defined type, with access specifiers. This is what distinguishes a C++ program from a C program.
6. **Function declaration (prototype) section** — signatures of user-defined functions, `return-type name(argument types);`, so the compiler can check calls made before the body is seen.
7. **Main() function section** — the **entry point**; execution begins here. Exactly one `main` per program; returns `int` (0 = success). Whatever else exists, the program is driven from here.
8. **Sub-declaration section** — the local declarations inside `main` before the statements: variables, objects, pointers; local scope, created on entry, destroyed on exit.
9. **Sub-program (function definition) section** — the bodies of the user-defined functions, usually placed after `main`; each is executed only when called.

```
// 1 Documentation : area of a circle
#include <iostream>            // 2 Link
#define PI 3.14159             // 2 Link (macro)
using namespace std;
const int DEC = 2;             // 3 Definition
int callCount = 0;             // 4 Global declaration
class Circle {                 // 5 Class declaration
    double r;
  public: Circle(double x) : r(x) {}  double area() const;
};
double areaOf(Circle c);       // 6 Function declaration
int main() {                   // 7 Main
    Circle c(2.0);             // 8 Sub-declaration
    cout << areaOf(c) << "\n";
    return 0;
}
double Circle::area() const { return PI * r * r; }   // 9 Sub-program definition
double areaOf(Circle c)      { ++callCount; return c.area(); }
```

Verified output: `12.5664` — and this same question returns as a **10-marker** in 2025 Q.5(C), so the section names are worth memorising verbatim.

Q.2 (B) Create a program to demonstrate the use of call by value. [6]

Call by value copies the **value** of the argument into the function's formal parameter. The parameter is a fresh local variable at a different address, so changes inside the function are **not visible** to the caller. Default mechanism in C++; safe but costs a copy (and for class objects invokes the copy constructor). The alternative, **call by reference** (`void f(int &x)`), passes an alias, so changes do affect the caller; pointers (`void f(int *p)`) achieve the same through an explicit address.

```
#include <iostream>
using namespace std;
void swapByValue(int x, int y) {          // receives copies only
    int t = x; x = y; y = t;
    cout << "Inside function: a=" << x << "  b=" << y << "\n";
}
int main() {
    int a = 10, b = 20;
    cout << "Before call  : a=" << a << "  b=" << b << "\n";
    swapByValue(a, b);
    cout << "After call   : a=" << a << "  b=" << b << "\n";
    return 0;
}
```

Verified output:

```
Before call  : a=10  b=20
Inside function: a=20  b=10
After call   : a=10  b=20
```

The point of the demonstration is the third line: inside the function the values really did swap, but the caller's `a` and `b` are untouched — proof that only a copy was passed. Finish the answer by stating the drawback (a big object is copied) and the fix (`&` reference, or `const &` when the function only reads).

Q.2 (C) Explain friend function with example. [10]

A **friend function** is a **non-member** function that is granted permission to access the **private and protected** members of a class. It is declared inside the class with the keyword `friend` (that declaration may sit in the private, protected or public part — position makes no difference) and is **defined outside the class like an ordinary function, without the class-name qualifier and without a `this` pointer**.

Why it exists: C++ protects data strictly, yet some operations naturally involve **two different classes** (comparing two objects, overloading a binary operator with the left operand not being an object). A member function of one class cannot see the internals of the other, so friendship is the controlled exception.

Properties to list 1. Not in the class's scope; **cannot be called with the dot operator** — invoke it as `show(d)` not `d.show()`. 2. Has **no `this`** pointer, so it must be passed the object(s) explicitly. 3. Declared with `friend` inside the class, defined outside; may also be declared before/after the class as long as the class is visible. 4. Breaking **encapsulation is deliberate and limited** — the granting class decides exactly which function it trusts. 5. Friendship is **not inherited**, **not transitive** (a friend of my friend is not my friend), and is **not mutual** automatically. 6. Usually passed objects **by const reference** to avoid copying, and to allow temporary objects. 7. **Friend class**: `friend class Tester;` gives every member of `Tester` the same access. 8. A member function of class A can be made a friend of class B (declare it in B after defining A).

Syntax

```
class ClassName {
    private:  int a;
    public:   void set(int x) { a = x; }
    friend  void friendFn(ClassName &c);      // declaration, no class qualifier
};
void friendFn(ClassName &c) {                  // ordinary function definition
    cout << "private a = " << c.a << "\n";     // legal - it is a friend
}
```

Complete program

```
#include <iostream>
using namespace std;
class Distance {
    int metre;                                  // private
  public:
    Distance() : metre(5) {}
    void setMetre(int m) { metre = m; }
    friend int sumDistance(Distance d1, Distance d2);
    friend void show(Distance d);
};
int sumDistance(Distance d1, Distance d2) { return d1.metre + d2.metre; }
void show(Distance d) { cout << "Private metre reached by friend = " << d.metre << "\n"; }
int main() {
    Distance A, B;
    A.setMetre(11); B.setMetre(22);
    show(A);
    cout << "Sum of two distances = " << sumDistance(A, B) << "\n";
    return 0;
}
```

Verified output:

```
Private metre reached by friend = 11
Sum of two distances = 33
```

Where friendship is genuinely used: operator overloading with a non-class left operand (`friend ostream& operator<<(ostream&, const X&)`), conversions between two unrelated classes, and class-to-class comparison. **Merits:** enables I/O overloading, useful for functions needing two objects of the same class, does not force the callee to become a member. **Demerits:** it breaches data hiding, so overuse quietly destroys the encapsulation that made the class safe in the first place — the standard conclusion line.

Q.2 (C — OR branch) Explain virtual function with example. [10]

A **virtual function** is a member function declared `virtual` in the **base class** and **redefined (overridden)** in the derived class. It is the mechanism behind **runtime (dynamic) polymorphism**: which version runs is decided **at run time by the actual object the pointer/reference points to**, not at compile time by the pointer's declared type. That decision is made through the **vtable** (one virtual table per polymorphic class, one **vptr** per object) — which is precisely **late binding**.

Why it is needed. Without `virtual`, a base-class pointer calling a function that the derived class also defines resolves to the **base** version (early binding) — the object's real type is ignored. With `virtual`, the call is dispatched to the derived version, which makes generic code possible: an array of `Shape*` can call `draw()` and each element draws itself.

Rules * Declared with `virtual` in the base; the `override` keyword in the derived class is optional in C++98 and **recommended** from C++11 because the compiler then errors if the signature does not actually match. * Signature (name, parameter types, const-ness, return type) must match exactly — a differing signature **overloads** instead of overriding. * Accessed **only through a pointer or reference** to the base class; calling on a plain object is resolved statically. * Constructors **cannot** be virtual; destructors **should** be virtual whenever the class is used polymorphically (otherwise `delete` on a base pointer skips the derived destructor and leaks). * A class with at least one virtual function has a vptr; `virtual void f() = 0;` makes it **pure virtual**, turning the class into an **abstract class** that cannot be instantiated and forcing every derived class to implement it. * `using` can bring a base version back into scope; a call can be forced statically with `Base::f()`.

```
#include <iostream>
using namespace std;
class Base {
  public:
    virtual void display() { cout << "Display of Base class\n"; }
    void normal()          { cout << "Non-virtual: always Base version\n"; }
};
class Derived : public Base {
  public:
    void display() override { cout << "Display of Derived class\n"; }
};
int main() {
    Base *p; Derived d; Base b;
    p = &b;  p->display();     // Base    - object is a Base
    p = &d;  p->display();     // Derived - late binding
    p->normal();               // base version: not virtual
    return 0;
}
```

Verified output:

```
Display of Base class
Display of Derived class
Non-virtual: always Base version
```

The third line is the whole lesson: `p` points at a `Derived`, yet `normal()` still prints the Base text, because only the **virtual** function is dispatched dynamically.

Q.3 (A) Create a program to demonstrate the use of class and object. [4]

```
#include <iostream>
#include <string>
using namespace std;
class Student {                  // class = blueprint, no memory yet
    int    roll;                 // private by default
    string name;
  public:
    void accept(int r, string n) { roll = r; name = n; }
    void display() const         { cout << "Roll: " << roll << "  Name: " << name << "\n"; }
};
int main() {
    Student s1, s2;              // two objects, each with its own copy of roll/name
    s1.accept(101, "Bhavik");
    s2.accept(102, "Aarav");
    s1.display();
    s2.display();
    return 0;
}
```

Verified output:

```
Roll: 101  Name: Bhavik
Roll: 102  Name: Aarav
```

Marks are for: `class` keyword and the trailing `;`, data members in the **private** part, member functions in `public`, objects created in `main`, access via the **dot** operator, and showing that `roll`/`name` are **independent per object**. Note that `s1.roll = 5;` from `main` would be a **compile error** — that is the encapsulation demonstration, so mention it explicitly.

Q.3 (B) Explain default constructor with example. [6]

A **default constructor** is a constructor that can be called **with no arguments**. Two legal forms: 1. One the compiler **synthesises** when you declare no constructor at all — it merely creates the objects and performs **no initialisation**, leaving built-in members with garbage (indeterminate) values. 2. One **you write** with either no parameters or all parameters carrying default values — used to give members a sensible starting state.

```
#include <iostream>
#include <string>
using namespace std;
class Item {
    int id; float price; string name;
  public:
    Item() {                                  // user-written default constructor
        id = 0; price = 0.0f; name = "not-set";
        cout << "Default constructor called\n";
    }
    void show() const { cout << "id=" << id << " name=" << name << " price=" << price << "\n"; }
};
int main() {
    Item i1;                                  // invokes Item() automatically
    i1.show();
    return 0;
}
```

Verified output:

```
Default constructor called
id=0 name=not-set price=0
```

Points to write: invoked **implicitly and exactly once per object** at the moment of creation; **no return type, not even** `void`; same name as the class; usually `public`; may be **overloaded** (a class can have several constructors, including one that is callable with zero arguments because its parameters have defaults). Two classic traps worth naming: `Item i2();` declares a **function**, not an object (the "most vexing parse") — the correct no-argument definition is `Item i2;`; and a default constructor is mandatory for array-of-objects declarations (`Item box[10];`) and for base classes whose derived constructors do not name it explicitly. Since C++11, `Item() = default;` asks the compiler for its version explicitly.

Q.3 (C) Explain function overloading with example. [10]

Function overloading is declaring **two or more functions in the same scope with the same name but a different parameter list**. It is a form of **compile-time (static) polymorphism**: the compiler picks the right one from the argument types and count at the call site, so one meaningful name ("area") can serve several shapes.

What makes an overload valid — the signature is the parameter list: * Different **number** of parameters ✓ · different **types** ✓ · different **order** of types ✓ * Different return type **only** ✗ (error — return type is **not** part of the signature) * Different parameter **names** only ✗ * Adding `const` to a **by-value** parameter ✗; `const` on a reference/pointer parameter ✓ * `default` arguments cannot be used to distinguish overloads — `f(int)` and `f(int, int = 10)` create an **ambiguity** when called as `f(5)`.

How the call is resolved: the compiler first looks for an **exact match**; failing that it tries **promotions** (`char` → `int`); failing that **standard conversions** (`int` → `double`); only then **user-defined conversions**. A call matching two candidates equally is an ambiguity error.

```
#include <iostream>
using namespace std;
class Area {
  public:
    double area(double r)           { cout << "Circle   : "; return 3.14159 * r * r; }
    double area(double l, double w) { cout << "Rectangle: "; return l * w; }
    double area(int b, int h)       { cout << "Triangle : "; return 0.5 * b * h; }
};
int main() {
    Area a;
    cout << a.area(2.0)      << "\n";
    cout << a.area(4.0, 5.0) << "\n";
    cout << a.area(6, 3)     << "\n";
    return 0;
}
```

Verified output:

```
Circle   : 12.5664
Rectangle: 20
Triangle : 9
```

Operator overloading is function overloading applied to operators (see Q.4(C)); **constructor overloading** works the same way — `File()`, `File(int)`, `File(const File&)` in the program above. Merits: readability (one concept, one name), convenience, cleaner interfaces, type-specific behaviour without forcing the caller to remember four names. Demerits: overuse confuses readers, resolution rules can surprise, and you cannot overload on return type.

Q.3 (A — OR branch) Design the C++ stream class structure. [4]

All C++ I/O is built on the class **`ios`**, which is derived (virtually) from `ios_base`; `istream` and `ostream` derive from `ios`, and the file and string streams derive from those. The standard objects `cin`, `cout`, `cerr`, `clog` are pre-created objects of these classes.

```
                  ios_base             (formatting flags, no templates)
                      |
                     ios               (holds the streambuf pointer, state bits)
             /      |    \     \
        basic_istream  basic_ostream  basic_iostream      (bidirectional)
            /   \         /    \         /
     ifstream  istream  ofstream ostream  iostream
       (file)   |         (file)  |        (file, both)
              wistream/wostream ... (wide-character family: wcin, wcout)

  Predefined objects:  cin : istream      cout : ostream
                       cerr : ostream (unbuffered, errors)
                       clog : ostream (buffered, errors)
```

- **`ios`** — base of the hierarchy; maintains formatting state and error flags (`goodbit`, `eofbit`, `failbit`, `badbit`) and ties the stream to a **`streambuf`** that actually moves bytes.
- **`istream`** — formatted/extracted **input**: `>>`, `get`, `getline`, `read`, `peek`, `ignore`.
- **`ostream`** — **output**: `<<`, `put`, `write`, `flush`.
- **`iostream`** — both, used for `in|out` file streams.
- **`ifstream`** / **`ofstream`** / **`fstream`** — file input, file output, file input-output respectively.
- **`istringstream`** / **`ostringstream`** / **`stringstream`** — read/write to a `string` in memory.

- The classes are templates over a character type (`basic_istream<charT>`); `istream` is the `char` typedef, `wistream` the `wchar_t` one.

Q.3 (B — OR branch) Explain static data member with example. [6]

A **static data member** is a class member **shared by all objects of the class** rather than copied into each object. There is **exactly one copy**, stored separately (not inside any object), created before `main` begins and existing for the whole program — so it is the class-level equivalent of a global variable, but kept inside the class's scope and access control.

Rules to state: declared inside the class with `static`; **must be defined and initialised outside the class**, `type ClassName::name = value;`, normally in one translation unit or you get an unresolved-symbol link error (in-class initialisation with `static constexpr` is the modern exception); accessible through an object (`e.count`) **or the class name** (`ClassName::getTotal()`, required for a static member function); a **static member function** has no `this`, so it may touch only static members — that is why `getTotal()` below is static too; `private` static members are reachable only by friends and members.

```
#include <iostream>
#include <string>
using namespace std;
class Employee {
    static int count;                // declaration - shared by every object
    string name;
  public:
    Employee(string n) : name(n) { ++count; }
    static int getTotal() { return count; }
    void show() const { cout << name << " | employees so far = " << count << "\n"; }
};
int Employee::count = 0;             // definition - mandatory, outside the class
int main() {
    Employee e1("Aarav"), e2("Bhavik"), e3("Meera");
    e1.show(); e2.show(); e3.show();
    cout << "Total via class name (no object) = " << Employee::getTotal() << "\n";
    return 0;
}
```

Verified output:

```
Aarav | employees so far = 3
Bhavik | employees so far = 3
Meera | employees so far = 3
Total via class name (no object) = 3
```

Note what the output proves: all three objects report **3**, not 1, 2, 3 — a single shared cell, incremented by each constructor. Uses: object counters, a shared configuration, the singleton pattern, values common to all instances. Contrast with an ordinary member (one copy per object, memory proportional to object count).

Q.3 (C — OR branch) Explain function overriding with example. [10]

Overriding is redefining, in the **derived class**, a member function that already exists in the **base class with exactly the same signature** — same name, same parameter types, same return type (covariant returns allowed), same `const` qualification. The derived version then **replaces** the inherited one for objects of the derived class.

Overriding vs overloading — the distinction the marker wants:

	Overloading	Overriding
Where	Same class (or scope)	Base **and** derived class
Signature	Same name, **different** parameters	Same name, **identical** parameters
`virtual`	Not involved	Base function **must be** `virtual`
Binding	**Early / static**, at compile time	**Late / dynamic**, at run time via the vtable
Purpose	One name for related operations	Redefine inherited behaviour

Hiding, the third case: redefining a base function **without** `virtual` is name **hiding** — the derived class's own calls see the new version, but a base-class pointer still reaches the base version. This is exactly what the program below demonstrates, so write the two outcomes side by side.

```
#include <iostream>
using namespace std;
class Parent {
  public:
    void show()           { cout << "Parent::show()\n"; }
    virtual void voice()  { cout << "Parent::voice()\n"; }
};
class Child : public Parent {
  public:
    void show()            { cout << "Child::show()   (hides parent, early binding)\n"; }
    void voice() override  { cout << "Child::voice()  (overrides, late binding)\n"; }
};
int main() {
    Parent *p; Child c;
    p = &c;
    p->show();      // Parent version - not virtual
    p->voice();     // Child version  - virtual
    return 0;
}
```

Verified output:

```
Parent::show()
Child::voice()  (overrides, late binding)
```

Add the qualifying points: use the `override` specifier so a signature typo becomes a compile error instead of a silent overload; call the base version deliberately with `Parent::voice()`; a **pure virtual** (`= 0`) base function *must* be overridden before the derived class can be instantiated; the base **destructor must be virtual** or deleting through a base pointer is undefined behaviour; overriding is what makes a **generic interface** possible — one `Shape*` collection, one `draw()` call, as many behaviours as there are classes.

Q.4 (A) Explain the scope resolution operator. [4]

The operator `::` (two colons) is a **binary operator that qualifies a name with the scope in which it lives**, resolving ambiguity where the same identifier exists at several levels.

Four uses, each worth a line in the answer 1. **Reveal a hidden global variable** when a local one shadows it:

```
int x = 10;                     // global
int main() { int x = 5;
    cout << x << " " << ::x;    // 5 (local) and 10 (global)
}
```

2. **Define a member function outside its class**:

```
class Circle { double r; public: double area() const; };
double Circle::area() const { return 3.14 * r * r; }
```

3. **Name a static member without an object**: `Employee::getTotal()`, `int Employee::count = 0;`.
4. **Reach into a namespace or resolve ambiguity**: `std::cout`, and with multiple inheritance `Derived::Base1::show()` when both bases declare `show()`. Also the pointer-to-member forms `.*` and `->*`, and global-scope prefix `::operator new`.

Q.4 (B) Create a program to convert feet to inches. [6]

1 foot = 12 inches, so `inches = feet * 12`. Demonstrates function definition, prototype, `float` arguments and the value-returning call.

```
#include <iostream>
using namespace std;
float feetToInch(float ft);                    // prototype (function declaration)
int main() {
    float feet;
    cout << "Enter feet: ";
    cin  >> feet;                              // run-time input
    cout << feet << " feet = " << feetToInch(feet) << " inches\n";
    return 0;
}
float feetToInch(float ft) {                   // definition
    return ft * 12.0f;
}
```

Verified output (input `5.5`):

```
Enter feet: 5.5 feet = 66 inches
```

Marks go to: correct conversion factor, a function that **takes a value and returns a value**, prototype separated from definition, and the call inside `cout`. Variants the setter may spring: feet-and-inches to inches total (`feet*12 + inch`), or inches back to feet-and-remainder using `/` and `%` — the same skeleton works for both.

Q.4 (C) Explain operator overloading in brief with example. [10]

Operator overloading lets an existing C++ operator be **redefined for user-defined types**, implemented as a special function whose name is the keyword `operator` followed by the symbol: `operator+`. It is **compile-time polymorphism** — one symbol, several meanings, chosen by operand types — and it makes classes feel like built-in types (`c1 + c2` instead of `c1.add(c2)`).

Rules — write all of these, they are where the marks are * Only **existing** operators can be overloaded; you cannot invent `operator**`, and cannot overload `::` `.` `.*` `?:` `_Nullable` (and `sizeof`). * **Arity is preserved** except for the new-style `operator()`; a binary operator overloaded as a member takes **one** parameter (the left operand is `*this`), as a friend it takes **two**. * **At least one operand must be a user-defined type** — you cannot redefine `int + int`. * **Precedence, associativity and grammar do not change**; the meaning of `a + b * c` stays the same even after overloading. * **Cannot be overloaded on return type alone**; the built-in meaning for fundamental types is never lost. * **Assignment `=`, `&`, `,`, and function-call `()` are overloadable; `[]` gives subscript access**; `<<` and `>>` must be overloaded as **friend** functions (the left operand is the stream, not the object). * Unary as member takes **no** argument; as friend

takes one. * If you overload `+`, provide `+=` too (and vice-versa conventions: symmetric operators best as non-member/friends; `<<`, `>>` must be).

```
#include <iostream>
using namespace std;
class Complex {
    double re, im;
  public:
    Complex(double r = 0, double i = 0) : re(r), im(i) {}
    Complex operator+(const Complex& c) const { return Complex(re + c.re, im + c.im); }
    Complex operator-(const Complex& c) const { return Complex(re - c.re, im - c.im); }
    bool    operator==(const Complex& c) const { return re == c.re && im == c.im; }
    Complex operator-() const                 { return Complex(-re, -im); }   // unary
    void show() const { cout << re << (im < 0 ? "-" : "+") << (im < 0 ? -im : im) << "i"; }
};
int main() {
    Complex c1(3, 4), c2(1, 2), c3;
    c3 = c1 + c2;
    cout << "c1 + c2 = "; c3.show(); cout << "\n";
    c3 = c1 - c2;
    cout << "c1 - c2 = "; c3.show(); cout << "\n";
    c3 = -c1;
    cout << "-c1     = "; c3.show(); cout << "\n";
    cout << "c1 == c2 ? " << (c1 == c2 ? "yes" : "no") << "\n";
    return 0;
}
```

Verified output:

```
c1 + c2 = 4+6i
c1 - c2 = 2+2i
-c1     = -3-4i
c1 == c2 ? no
```

The interesting line is the third: **the same symbol `-` is overloaded twice** in the class — once binary, once unary — which is legal because the arity differs. Typical applications: arithmetic on Complex/Matrix/Date, `cout <<` for user types, `[]` for smart pointers and containers, `()` for functors, `==` for comparing objects by value.

Q.4 (A — OR branch) List file modes and explain any two. [4]

File modes are flags of class `ios` (in `<fstream>`) passed to `open()` or to the stream constructor; they combine with `|`.

Mode	Meaning
`ios::in`	open for **reading**
`ios::out`	open for **writing** (default for `ofstream`)
`ios::ate`	**at end** — seek to end of file right after opening, then read/write from anywhere
`ios::app`	**append** — every write goes to the end, position cannot move backwards
`ios::trunc`	**truncate** — discard the existing contents if the file exists (default with `out`)
`ios::binary`	read/write in **binary** mode, no end-of-line or text translation

Explain two (a full-sentence version): * `ios::app` — the file is opened so that all output is written at the end, preserving what was already there; the put pointer is forced to the end before every write, so the existing data cannot be overwritten or lost. Used for log files. If the file does not exist it is created. Combines with `out`;

`app` + `in` is possible only for `fstream`. * **`ios::trunc`** — when opening with `out`, if a file of that name exists its contents are **erased** and the length becomes zero before writing starts; the default behaviour of `ofstream f("data.txt")`. Never combine `trunc` with `app` — the two contradict each other and the open fails. * (or) **`ios::binary`** — treats the file as a raw sequence of bytes, disabling text-mode translation of line endings and EOF handling, which is mandatory for `read()` / `write()` of objects, images and audio; without it, structure sizes and newline bytes get mangled.

```
ofstream out("report.txt", ios::app);                       // append
fstream  io("student.dat", ios::in | ios::out | ios::binary);
```

`fail()` / `is_open()` must be tested after opening; a file is closed with `close()` (or by the destructor).

Q.4 (B — OR branch) Explain the Standard Template Library in detail. [6]

The **STL** is a large, platform-independent **library of generic (template-based) components** shipped with every conforming C++ implementation. It is the canonical example of **generic programming**: algorithms written once against abstract requirements, working for any type and any container that satisfies them.

Four principal components (plus allocators) 1. **Containers** — template class **objects that store and organise collections of data**. Three families: * **Sequence containers** — `vector` (dynamic array, random access, amortised growth), `deque` (double-ended queue), `list` (doubly linked list, insert/erase anywhere in constant time), `forward_list` (singly linked), `array` (fixed size, C++11), plus the **adapters** `stack`, `queue`, `priority_queue` built on top of them. * **Associative containers** — ordered by a comparator, implemented as **red-black trees**: `set`, `multiset`, `map`, `multimap`; logarithmic lookup, unique keys for `set`/`map`. * **Unordered associative containers** (C++11) — **hash tables**: `unordered_set`, `unordered_map`, … average constant-time lookup. 2. **Algorithms** — ~90 non-member **function templates** in `<algorithm>` / `<numeric>` that operate on ranges `[first, last)`: **non-modifying** (`find`, `count`, `search`, `equal`, `for_each`), **mutating** (`copy`, `fill`, `swap`, `generate`, `transform`, `replace`), **removing** (`remove`, `unique`), **sorting & searching** (`sort`, `stable_sort`, `binary_search`, `merge`, `lower_bound`, `nth_element`), **set operations** (`set_union`, `set_intersection`), **min/max**. `sort` requires **random-access iterators**, so it does not work on `list`. 3. **Iterators** — the **glue**: objects that behave like smart pointers walking a container, providing `*it`, `++it`, `it->`; categories **input, output, forward, bidirectional, random-access** (and contiguous, C++17) determine which algorithms a container can accept. `begin()`, `end()`, `advance`, `distance`. 4. **Functors / function objects** — objects with `operator()` used as the callback argument of algorithms (predicates, comparators); C++11 adds lambdas, `std::function`, `bind`. 5. **Allocators** — `std::allocator<T>` abstracts how container memory is obtained; rarely touched directly.

```
#include <iostream>
#include <vector>
#include <algorithm>
int main() {
    std::vector<int> v = {5, 2, 9, 1};
    std::sort(v.begin(), v.end());                      // algorithm + iterators
    for (int x : v) std::cout << x << " ";              // 1 2 5 9
    auto it = std::find(v.begin(), v.end(), 9);
    std::cout << "\nfound 9 at index " << (it - v.begin()) << "\n";
}
```

Merits to finish with: code reuse and shorter development time; containers and algorithms tested and optimised; **type-safe** — errors caught at compile time, unlike C's `void*`; **high performance** because templates inline away (no run-time virtual dispatch on the container itself); extensible with your own iterators, comparators and allocators. **Demerit worth naming:** long, ugly compiler diagnostics from template instantiation (much improved by C++20 concepts).

Q.4 (C — OR branch) Explain multilevel inheritance with example. [10]

Multilevel inheritance is a **chain**: class A is the base of class B, and class B in turn is the base of class C — three or more levels deep. C inherits everything B and A made accessible (B's inherited members pass down as B's own), so **C has access to the non-private members of both A and B**, while B acts as both a derived and a base class. Distinguish from *multiple* inheritance (two parents on one level) and from *hierarchical* (one parent, many children).

Construction order is a common extra mark: constructors run **base → derived** (`A(), B(), C()`), destructors run in the **reverse** order (`~C(), ~B(), ~A()`).

```
#include <iostream>
#include <string>
using namespace std;
class Student {                       // level 1 - grandparent
  protected: int roll; string name;
  public: void getRoll(int r, string n) { roll = r; name = n; }
};
class Test : public Student {         // level 2 - parent
  protected: float m1, m2;
  public: void getMarks(float a, float b) { m1 = a; m2 = b; }
};
class Sports : public Test {          // level 3 - child
    int score;
  public:
    void getScore(int s) { score = s; }
    void display() {                    // uses roll, name, m1, m2 - all inherited
        cout << "Roll=" << roll << " Name=" << name
             << " Marks=" << m1 << "," << m2
             << " SportScore=" << score << "\n";
    }
};
int main() {
    Sports sp;
    sp.getRoll(101, "Bhavik");
    sp.getMarks(85, 91);
    sp.getScore(12);
    sp.display();
    return 0;
}
```

Verified output:

```
Roll=101 Name=Bhavik Marks=85,91 SportScore=12
```

`protected` is the key device: it keeps the data hidden from `main` yet lets it travel down the chain. Other exam points: with **private** inheritance the chain stops (grandchildren cannot reach it); each level adds its own members and can override an inherited one; a deep chain raises coupling — "prefer composition to inheritance" is the one-line critique. Real-life example: `Mammal → Primate → Human`, or `Vehicle → Car → ElectricCar`.

Q.5 (A) Explain re-throw. [4]

Re-throwing means **catching an exception and throwing the very same exception onward to an outer (calling) handler**, using a bare `throw;` with **no expression** inside a `catch` block.

```
void inner() {
    try {
        throw 42;
    }
    catch (int e) {
        cout << "logged, but cannot fix it\n";
        throw;                    // RE-THROW: same exception, same type, same object
    }
}
int main() {
    try { inner(); }
    catch (int e) { cout << "outer handler saw " << e << "\n"; }
}
```

Points that earn the marks * `throw;` **must appear inside a** `catch` **block** (or inside a function called from one) — outside a handler there is nothing to rethrow, and the program calls `std::terminate`. * It rethrows the **original exception object**, preserving its **dynamic type** — this is why the handler should take `catch (const Base& e)` rather than by value. * Contrast `throw e;`: that **throws a new copy**, and if `e` was declared as a base class, the copy is sliced to the base type, so an outer `catch (Derived&)` would **miss** it. This is the classic reason for using bare `throw;`. * Purpose: **partial handling** — a function cleans up or logs what it can see, then hands the problem upward to whoever actually knows how to deal with it. It supports the "handle or propagate, not both blindly" design rule. * Since C++11 the state used by `throw;` is the *currently-handled exception*; a `std::rethrow_exception` (with `std::exception_ptr`) is the explicit-storage variant.

Q.5 (B) Explain class template with example. [6]

A **class template** is a **generic class** — a blueprint with one or more **type parameters**, from which the compiler manufactures concrete classes on demand. Each distinct set of arguments produces a **different class** (template instantiation), so `Array<int,4>` and `Array<double,3>` are two unrelated types generated from one source. It is compile-time polymorphism, the foundation of the STL (`vector<T>`, `map<K,V>`).

Syntax

```
template <class T>  class Stack { ... };        // type parameter
template <typename T, int N> class Array { ... }; // type + non-type parameter
template <class T = double> class Box { ... };   // default argument (C++11)
```

Using it: `Stack<int> s;` — the compiler infers nothing at declaration in C++98; from C++17 `Stack s{1};` deduces `T` (CTAD).

```
#include <iostream>
using namespace std;
template <class T, int size>
class Array {
    T arr[size];
  public:
    void fill() { for (int i = 0; i < size; ++i) arr[i] = T(i + 1); }
    void show() const { for (int i = 0; i < size; ++i) cout << arr[i] << " "; cout << "\n"; }
    T total() const { T s = T(); for (int i = 0; i < size; ++i) s += arr[i]; return s; }
};
template <typename T>
void swapValues(T &a, T &b) { T t = a; a = b; b = t; }    // function template for contrast
int main() {
    Array<int, 4> ai;
    ai.fill();
    cout << "Array<int,4>   : "; ai.show();
    cout << "total          : " << ai.total() << "\n";
    int x = 1, y = 2; double p = 1.5, q = 2.5;
    cout << "before swap ints: " << x << " " << y << "\n";
    swapValues(x, y);
    cout << "after  swap ints: " << x << " " << y << "\n";
    swapValues(p, q);
    cout << "after  swap dbls: " << p << " " << q << "\n";
    return 0;
}
```

Verified output:

```
Array<int,4>   : 1 2 3 4
total          : 10
before swap ints: 1 2
after  swap ints: 2 1
after  swap dbls: 2.5 1.5
```

One source served `int` and `double` with no duplicated code, and `swapValues` worked for both — that is the whole argument for templates. Drawback to name: **all members are compiled only when used**, so errors surface far from their cause and diagnostics are long; the definition normally has to live in the header.

Q.5 (C) Explain user-defined exception with example. [10]

C++ exception handling transfers control from the point of failure to a **handler**: `try` encloses risky code, `throw` raises an object, `catch` matches by **type** and recovers. The matching walks **up the call stack** looking for a compatible handler; `catch (...)` catches anything; derived exception classes must be caught **before** their bases; and the parameter is conventionally `const X&` to avoid slicing and copying.

User-defined exceptions are exception classes **you declare yourself**, normally deriving from `std::exception` (which supplies `virtual const char* what() const noexcept`) or from another exception type, so they can carry **application-specific detail** — the offending value, the field name, an error code.

```
#include <iostream>
#include <exception>
using namespace std;
class AgeOutOfRange : public exception {      // user-defined exception class
    int value;
  public:
    AgeOutOfRange(int v) : value(v) {}
    const char* what() const noexcept override { return "Age must be between 0 and 120"; }
    int getValue() const { return value; }
};
class NegativeSalary : public exception {
  public:
    const char* what() const noexcept override { return "Salary cannot be negative"; }
};
int main() {
    try {
        int age = 130, salary = -5000;
        if (age < 0 || age > 120) throw AgeOutOfRange(age);
        if (salary < 0)           throw NegativeSalary();
    }
    catch (const AgeOutOfRange& e)  { cout << "Caught: " << e.what() << " (offending value " << e.getValue()
<< ")\n"; }
    catch (const NegativeSalary& e) { cout << "Caught: " << e.what() << "\n"; }
    catch (...)                     { cout << "Unknown exception\n"; }
    cout << "Program continued normally\n";
    return 0;
}
```

Verified output:

```
Caught: Age must be between 0 and 120 (offending value 130)
Program continued normally
```

Note what the output shows: only the **first** throw runs (control leaves the `try` block immediately, so the salary check never happens), each specific handler is reached by type, and execution **continues after** the whole `try` / `catch` — not inside the aborted block.

Structure of the full answer 1. **Keywords** — `try`, `throw`, `catch`; the order of execution: throw → search handler → unwind (destructors of destroyed automatic objects run) → catch → resume after the try. 2. **Why not an error code** — separation of normal and error paths, cannot be ignored accidentally, propagates automatically across stack frames, and objects rather than codes let you bundle context. 3. **Standard hierarchy** — `exception` → `logic_error` (`invalid_argument`, `out_of_range`, `domain_error`) and `runtime_error` (`range_error`, `overflow_error`); `bad_alloc` from `new`, `bad_cast`, `ios_base::failure`. 4. **Rules** — anything can be thrown (int, string, object); matching is by type, and a derived object matches a base-class handler; `noexcept` promises not to throw; a re-throw is `throw;` (Q.5(A) above); uncaught exceptions call `std::terminate`; resource safety relies on destructors (**RAII**) because `delete` after a throwing call is skipped. 5. **`throw` vs `return`** — throw unwinds out of any number of frames, return only one. 6. Merits: cleaner code, forced error handling, separation of concerns, error reporting with rich data. Demerits: run-time cost of unwinding, harder to trace control flow, exceptions must be documented or callers guess.

Q.5 (A — OR branch) Explain binary files in detail. [4]

A **binary file** stores data **exactly as it is held in memory** — a raw sequence of bytes of the object's representation — whereas a **text file** stores a human-readable character form of each value, with number-to-string conversion and newline translation.

```
#include <iostream>
#include <fstream>
using namespace std;
struct Student { int roll; float cgpa; };
int main() {
    Student s1{101, 8.5f}, s2;
    { ofstream out("stud.dat", ios::out | ios::binary);
      out.write((char*)&s1, sizeof s1); }                // write raw bytes
    { ifstream in("stud.dat", ios::in | ios::binary);
      in.read((char*)&s2, sizeof s2); }
    cout << "read back: roll=" << s2.roll << " cgpa=" << s2.cgpa << "\n";
}
```

Verified output: `read back: roll=101 cgpa=8.5`

Points that earn the marks: open with `ios::binary`; use the **block functions** `read()` and `write()` (casting the address to `char*` and passing `sizeof object`) instead of `<<` / `>>`; **no conversion** takes place, so it is much **faster** and saves space, and the file is **not human-readable** and not portable across machines with different endianness, `sizeof`, or padding; random access works with `seekg` / `seekp` (so records can be updated in place); suitable for images, audio, databases and object persistence; text mode on the other hand is portable and editable, and translates `\n` to CRLF on Windows.

Q.5 (B — OR branch) What is the use of `setw`? Explain with example. [6]

Manipulators are **special functions used with the insertion (`<<`) and extraction (`>>`) operators to change the format of I/O without touching the stream's flags by hand**. They live in `<iomanip>` (`setw`, `setprecision`, `setfill`, `setbase`, `setiosflags`, `resetiosflags`) and in `<ios>` (`endl`, `flush`, `ends`); the older `<iomanip.h>` / `<IOMANIP>` style of manipulators is the legacy pre-standard form.

`setw(n)` sets the **field width of the next item only** — the output is padded (with spaces by default) so it occupies at least `n` columns; if the value is longer than `n`, the field simply **expands and nothing is truncated**. Right alignment is the default, so `left` or `right` is combined with it, and `setfill('*')` chooses the pad character.

```
#include <iostream>
#include <iomanip>
using namespace std;
int main() {
    cout << setw(10) << "Name"   << setw(8) << "Marks" << "\n";
    cout << setw(10) << "Bhavik" << setw(8) << 85.5    << "\n";
    cout << setw(10) << "Meera"  << setw(8) << 9.25    << "\n";
    double pi = 3.14159265;
    cout << "default       : " << pi << "\n";
    cout << setprecision(3) << "setprecision(3): " << pi << "\n";
    cout << fixed << setprecision(4) << "fixed dp 4   : " << pi << "\n";
    cout << setw(12) << left << "cat" << "end\n";
    cout << "endl ends line" << endl << "next line\n";
    return 0;
}
```

Verified output (the column alignment is the point):

```
      Name   Marks
    Bhavik    85.5
     Meera    9.25
default       : 3.14159
setprecision(3): 3.14
fixed dp 4   : 3.1416
cat         end
endl ends line
next line
```

Two behaviours to state explicitly: `setw` is **not sticky** (each new column needs its own `setw`), whereas `setprecision`, `fixed` and `left` **persist** until changed. On **input**, `setw(n)` limits extraction to n−1 characters plus the terminating null, which is how `cin >> setw(20) >> buffer` prevents overflow.

Q.5 (C — OR branch) Explain exception handling and develop a C++ program to handle "Exception Division by Zero". [10]

Exception handling is the **run-time mechanism for detecting, reporting and recovering from errors** by separating the code that *can fail* from the code that *decides what to do*. Three keywords: `try` encloses the statements that may fail; `throw` raises an object (any type) when the error condition is detected — execution leaves the `try` block immediately; `catch` declares a parameter of the type it handles and runs the recovery. The search for a matching handler proceeds outwards through the caller chain; if none is found, `std::terminate` ends the program. Destructors of fully constructed automatic objects run during this unwinding, which is the basis of **RAII**.

```
#include <iostream>
#include <stdexcept>
using namespace std;
int divide(int a, int b) {
    if (b == 0) throw runtime_error("Exception Division by Zero");
    return a / b;
}
int main() {
    int a = 10;
    try {
        cout << a << " / 0 = " << divide(a, 0) << "\n";      // throws
    } catch (const runtime_error& e) {
        cout << "Error caught: " << e.what() << "\n";
    }
    try {
        cout << a << " / 2 = " << divide(a, 2) << "\n";      // normal
    } catch (const runtime_error& e) { cout << e.what() << "\n"; }
    cout << "Rest of program still executes\n";
    return 0;
}
```

Verified output:

```
10 / 0 = Error caught: Exception Division by Zero
10 / 2 = 5
Rest of program still executes
```

Why the demonstration matters: without the handler, dividing by zero on most platforms crashes the program (integer division by zero is **undefined behaviour**); with it the program **reports the problem and keeps running**, which is exactly the last line of output. Points to add for the full 10: `throw` may be a plain object, and `catch (const exception&)` catches anything derived from `std::exception`; several `catch` blocks may follow one `try` and are matched top-down, so **put derived types first**; `catch(...)` is the catch-all; a function can

be declared `noexcept` to promise it never throws (breaking the promise calls `terminate`); re-throwing with bare `throw;` passes the same object upward; and exceptions should be used for genuinely exceptional conditions, not as a substitute for ordinary control flow — the cost of unwinding makes that a performance issue as well as a readability one.

---

4. Winter Examination 2025 — full solved paper

Date 09/12/2025 · 3 hours · 100 marks

Q.2 (A) Draw diagrams illustrating: (I) Single inheritance (II) Multilevel inheritance. [4]

Single inheritance — one derived class from **one** base class; the simplest form, an "is-a" link with exactly one parent.

```
   [ Base : A ]            e.g.  [ Employee ]
        |                             |
        v                             v
   [ Derived : B ]                [ Manager ]
                                    inherits name, id
```

```
class A { public: int x; };
class B : public A { public: int y; };   // B has x AND y
```

Multilevel inheritance — a **chain of three or more** classes: A → B → C, where B is derived from A and C from B. C inherits from B, and through B it also inherits everything A made visible.

```
   [ Class A ]      (Student: roll, name)
        |
        v
   [ Class B ]      (Test: marks)          B is derived AND base
        |
        v
   [ Class C ]      (Sports: score)        C reaches A's and B's members
```

```
class A { protected: int a; };
class B : public A { protected: int b; };
class C : public B { public: void show() { cout << a << b; } };
```

For full marks label the arrows top-to-bottom, mark which members are `protected`, and state the construction order for multilevel: `A() → B() → C()`, destruction `~C() → ~B() → ~A()`. Contrast one line each with **multiple** (two parents, one child), **hierarchical** (one parent, several children) and **hybrid** (a combination, e.g. the diamond).

Q.2 (B) Explain the role of `public`, `private` and `protected` when deriving a subclass through inheritance. [6]

The **access specifier written in the derived-class declaration** (`class D : public B`) does **not** change the base class's own members — it **re-labels the inherited access**, deciding how the base's members appear inside `D` and to `D`'s own users. The base's `private` members are never directly accessible in the derived class under any mode, though they still exist inside each object.

Base member is	`public` derivation	`protected` derivation	`private` derivation
`public`	stays **public** in D	becomes **protected** in D	becomes **private** in D
`protected`	stays **protected** in D	stays **protected** in D	becomes **private** in D
`private`	**not accessible** in D (only via D's inherited public/protected functions)	not accessible in D	not accessible in D

Reading the table: `public` inheritance is by far the most common because it preserves the interface — **"a D *is a* B"**, so a `D*` can be used wherever a `B*` is expected (that substitution actually requires public derivation). `protected` inheritance hides the base interface from the world while letting further-derived classes keep using it. `private` inheritance means **"D is implemented in terms of B"** — the base interface is sealed off and no longer usable through a base pointer; composition is usually the better answer to the same need.

Also required for full marks: what each specifier means *inside a class* — `private` (default for `class`) reachable only by that class's own members and `friend`s, invisible to derived classes and to `main`; `protected` reachable by that class, its friends **and its derived classes**, but not by `main`; `public` reachable everywhere. Note `struct` defaults to `public`, `class` defaults to `private`; a `using Base::method;` declaration in the derived class can deliberately reopen a hidden base member; and a base **private** member can still be *obtained* through the public/protected accessor functions the base provides, which is the recommended design.

Q.2 (C) Explain templates and compare function templates with class templates with syntax. [10]

A **template** parameterises code by **type** (and by constant values) so a single definition serves many types — **generic programming** and a form of **compile-time polymorphism**. The compiler performs **instantiation**: it generates a separate concrete function or class for every distinct argument list encountered. Nothing is generated for types never used, so there is no run-time cost, and all type checking is done at compile time (unlike `void*`, which is unchecked).

Function template — one algorithm, many argument types.

```
template <typename T>              // or <class T>
T maximum(T a, T b) { return (a > b) ? a : b; }

maximum(3, 7);                     // T deduced as int      -> maximum<int>
maximum(2.5, 1.5);                 // T deduced as double
maximum<int>(10, 20);              // explicit argument
```

Multiple parameters are allowed (`template <class T, class U>`), as are **non-type** parameters (`template <int N>`) and default type arguments. Overloading a template with a plain function is legal; when both match exactly, the **non-template function wins**.

Class template — one container/abstraction, many element types.

```
template <class T, int N>
class Array {
    T arr[N];
  public:
    void fill()  { for (int i = 0; i < N; ++i) arr[i] = T(i + 1); }
    T total() const { T s = T(); for (int i = 0; i < N; ++i) s += arr[i]; return s; }
    void show() const { for (int i = 0; i < N; ++i) cout << arr[i] << " "; cout << "\n"; }
};

Array<int, 4> a;                   // must supply the arguments at the point of use
```

Member functions of a class template are themselves templates and are usually written outside as `template <class T, int N> T Array<T,N>::total() const { ... }`. A **specialization** (`template <> class Array<bool,4> {...}`) supplies a tailor-made version for one argument list.

Comparison — the part the marks are on

Defines	a family of **functions**	a family of **classes (types)**
Instantiated	when it is **called**	when an **object is declared**
Type argument	usually **deduced** from the arguments	must be **stated explicitly** (`Array<int,4>`) — except C++17 CTAD
Purpose	same logic for different data types	same structure/behaviour for different element types
Example from the standard library	`std::max`, `std::swap`	`std::vector<T>`, `std::map<K,V>`, `std::pair`
Overloading/specializing	can be overloaded with ordinary functions	can be partially or fully specialized
Non-type parameters	allowed	allowed (and commonly used for sizes)

```
#include <iostream>
using namespace std;
template <class T, int size>
class Array {
    T arr[size];
  public:
    void fill() { for (int i = 0; i < size; ++i) arr[i] = T(i + 1); }
    void show() const { for (int i = 0; i < size; ++i) cout << arr[i] << " "; cout << "\n"; }
    T total() const { T s = T(); for (int i = 0; i < size; ++i) s += arr[i]; return s; }
};
template <typename T>
void swapValues(T &a, T &b) { T t = a; a = b; b = t; }
int main() {
    Array<int, 4> ai;
    ai.fill();
    cout << "Array<int,4>   : "; ai.show();
    cout << "total          : " << ai.total() << "\n";
    int x = 1, y = 2; double p = 1.5, q = 2.5;
    cout << "before swap ints: " << x << " " << y << "\n";
    swapValues(x, y);
    cout << "after  swap ints: " << x << " " << y << "\n";
    swapValues(p, q);
    cout << "after  swap dbls: " << p << " " << q << "\n";
    return 0;
}
```

Verified output:

```
Array<int,4>   : 1 2 3 4
total          : 10
before swap ints: 1 2
after  swap ints: 2 1
after  swap dbls: 2.5 1.5
```

Closing points: advantages — reuse, type safety, performance (inlined, no virtual dispatch), and the STL depends on it; disadvantages — code bloat from many instantiations, slower compiles, and error messages that point inside the template rather than at your call.

Q.2 (C — OR branch) Discuss default arguments. Construct a program demonstrating different function calls using default argument values, and present the output. [10]

A **default argument** gives a parameter a value used **automatically when the caller omits that argument**. It lets one function serve several call forms, reducing the number of overloads and making the interface self-documenting.

```
void area(double length, double width = 5.0, char unit = 'm');
```

Rules — each is a mark * The default belongs to the **declaration**; writing it in both declaration and definition is a compile error. For a function with no declaration, the default must appear **before the first call**. * Defaults are filled **from right to left**: once a parameter has a default, **every parameter to its right must also have one**. `f(int a, int b = 2, int c)` is illegal. * The caller may omit only a **trailing** run of arguments — `area(4.0)` is fine, but you cannot skip `width` while supplying `unit` (a named-parameter workaround exists only in C++20, so say "positional skipping is impossible"). * **Overloaded functions cannot be distinguished by default arguments alone**: `f(int)` next to `f(int, int = 10)` makes `f(5)` ambiguous. * A default value must be a **constant expression, a global variable, or an expression not referring to a parameter of the same function**; a non-const reference/pointer cannot take a temporary as its default. * For a class member, the default goes in the **in-class declaration**; a redeclaration may not add another default for the same parameter. * Evaluation happens **at the call site**, in the caller's scope, once per call.

```
#include <iostream>
using namespace std;
void area(double length, double width = 5.0, char unit = 'm') {
    cout << "Area = " << length * width << " square " << unit << unit << "\n";
}
int main() {
    area(4.0);              // both defaults used   -> 4.0 * 5.0 m^2
    area(4.0, 3.0);         // unit defaulted       -> 4.0 * 3.0 m^2
    area(4.0, 3.0, 'c');    // no default used      -> 4.0 * 3.0 c^2
    return 0;
}
```

Verified output:

```
Area = 20 square mm
Area = 12 square mm
Area = 12 square cc
```

The three calls are exactly the demonstration asked for: **all defaults**, **one default**, and **no default**. Advantage: shorter code, one function instead of three overloads, easy evolution of an API without breaking old callers. Cost: the defaults are invisible at the call site, so behaviour can surprise readers, and defaults in a base class are not re-read by an overriding function (they are statically bound) — a subtle trap worth one line.

Q.3 (A) List any four real-life applications of Object-Oriented Programming. [4]

Any four, each with the class/objects that make it OOP: 1. **Banking systems** — `Account`, `Customer`, `Transaction` objects; balance is private, only `deposit`/`withdraw` may change it, and `Loan extends Account` reuses the ledger behaviour with different interest rules. 2. **E-commerce / retail** (the DBMS notes' commerce platform is the same shape) — `Product`, `Cart`, `Order`, `PaymentGateway` with polymorphic payment methods (UPI, card, wallet) behind one `pay()` interface. 3. **Reservation and ticketing** (airline, railway, cinema) — `Flight`, `Seat`, `Booking`; concurrency handled per seat object, `Passenger extends Person`. 4. **GUI / windowing toolkits and games** — `Window`, `Button`, `Dialog`, sprites: each widget is an object with state and its own `draw()`/`onClick()`, overridden by subclass, which is textbook polymorphism. 5. **Healthcare /**

hospital information systems — `Patient`, `Doctor`, `Appointment`, `Prescription` with privacy enforced through access control on the objects. 6. **Mobile and web applications, ERP, CAD/CAM and simulation** — real-world entities map naturally to classes; simulation of physical objects (vehicles, robots) is the classic OO case. 7. Also: multimedia information systems, expert systems/AI, object-oriented databases, and any large team project where modularity and reuse decide the schedule.

Q.3 (B) Differentiate between function overloading and function overriding. [6]

Both are **polymorphism**, but on opposite sides of compilation.

Basis		
Meaning	several functions with the **same name, different parameter list**	a derived class **redefines a base function with the same signature**
Where	**same class / same scope**	**base class and derived class** (never in one class)
Parameters	must differ in count, type or order	must be **identical**
Return type	may differ (not enough on its own)	must be the same (covariant types allowed)
`virtual` keyword	not involved	**required in the base** function
Binding	**Early / static**, decided at compile time	**Late / dynamic**, decided at run time through the vtable
Which object decides	the **argument types at the call**	the **actual (run-time) type of the object**
Inheritance involved	no	yes
Example	`print(int)`, `print(double)`, `print(int,int)`	`Shape::area()` redefined in `Circle::area()`
Purpose	convenience — one name, related operations	replacement — specialise inherited behaviour

```
class Shape { public: virtual double area() const { return 0; }   // overriding target
                      double scale(double f) { return f; }    // overloading set
                      double scale(double f, double g) { return f*g; } };
class Circle : public Shape { double r; public:
    double area() const override { return 3.14159 * r * r; } };   // override
```

One-line conclusion: overloading is resolved by *which arguments you passed*, overriding by *which object you are holding* — the first is compile-time, the second run-time. (Add that redefining a non-virtual base function is neither, it is **name hiding**, and a base pointer still reaches the base version.)

Q.3 (C) Explain typecasting in C++ and differentiate implicit and explicit casting with examples. [10]

Type casting (type conversion) is changing an expression of one data type into another, either automatically by the compiler or deliberately by the programmer. It is needed when mixing types in an assignment, an arithmetic expression, a function argument or a comparison, and it is the reason `7/2` prints `3` while `7.0/2` prints `3.5`.

1. Implicit (automatic / coercion / widening) conversion — done by the compiler with **no syntax from the programmer**, always in the *safe* direction (smaller → larger, or same rank), never losing magnitude: `char/short → int → unsigned int → long → float → double → long double`.

```
int    i = 7;
double d = i;              // int  -> double automatically
float  f = 10;             // int  -> float
cout << i + 3.5;           // int promoted to double, result 10.5
```

Three sub-cases worth naming: **integral promotion**, **usual arithmetic conversions** (both operands brought to a common type before the operator runs), and **implicit conversion in function calls** (`f(10)` where `f` takes a `double`). Implicit conversions are also created by any single-argument constructor, which is why `explicit` exists — marking the constructor `explicit` forbids that surprise.

2. Explicit (type casting / coercion by the programmer / narrowing) conversion — written by the programmer because the conversion is **lossy or unsafe**, so C++ demands an acknowledgement:

```
double d = 7.9;
int    i = (int)d;                  // C-style cast: i = 7, fraction discarded
int    j = static_cast<int>(d);     // C++ style, preferred
float  g = (float)7 / 2;            // 3.5 - prevents integer division
char   c = static_cast<char>(70);   // 'F'
```

C-style `(type)x` vs the four C++ named casts — a distinction that earns marks: `static_cast` (compile-time-checked conversions between related types, the everyday choice), `dynamic_cast` (**run-time**-checked downcast through pointers/references, needs polymorphic types, returns `nullptr` /throws `bad_cast` on failure), `const_cast` (adds or removes `const` / `volatile`), `reinterpret_cast` (reinterprets a bit pattern, e.g. pointer ↔ integer, machine-dependent and unsafe). The named casts are greppable, narrower in intent, and let the compiler help.

Comparison table

Basis		
Written by	compiler	programmer
Syntax	none	`(T)x` or `static_cast<T>(x)`
Direction	normally **widening** (safe)	usually **narrowing** (lossy)
Data loss	none	possible: `7.9 → 7`, overflow, precision
Safety	high	must be the programmer's responsibility
Example	`double d = 5;`	`int i = (int)5.9;`

```
#include <iostream>
using namespace std;
int main() {
    int i = 7; double d;
    d = i;                              // implicit int -> double
    cout << "implicit   d = i     : " << d << "\n";
    d = 7.9; i = (int)d;                // explicit double -> int, truncates
    cout << "explicit (int)d      : " << i << "\n";
    cout << "static_cast<int>(7.9): " << static_cast<int>(d) << "\n";
    cout << "7/2 without cast     : " << 7 / 2 << "\n";
    cout << "7/2 with cast        : " << (float)7 / 2 << "\n";
    return 0;
}
```

Verified output:

```
implicit   d = i     : 7
explicit (int)d      : 7
static_cast<int>(7.9): 7
7/2 without cast     : 3
7/2 with cast        : 3.5
```

The last two lines are the classic demonstration: **int/int is integer division and truncates before any conversion happens**, so casting one operand is what turns `3` into `3.5`. Also mention **type promotion in expressions** (`char` arithmetic is done as `int`) and the `explicit` keyword on constructors as the guard against unwanted implicit conversion.

Q.3 (A — OR branch) Identify the names and symbols of the two operators used with `cout` and `cin`. [4]

- `<<` **— the insertion (stream output put-back) operator**, used with `cout`: it **inserts** data into the output stream, sending it to the console. Overloaded for every built-in and user type, left-associative so it chains, and returns the stream itself, which is what makes chaining legal.
- `>>` **— the extraction operator**, used with `cin`: it **extracts** formatted data from the input stream into variables, stopping at whitespace, and returns the stream so it too chains.

```
#include <iostream>
using namespace std;
int main() {
    int roll; char name[20]; float cgpa;
    cout << "Enter roll, name, cgpa: ";          // insertion operator
    cin  >> roll >> name >> cgpa;                 // extraction operator, chained
    cout << roll << " " << name << " " << cgpa << endl;
    return 0;
}
```

Extra marks: `endl` inserts a newline **and flushes** the buffer; `cout` is buffered, `cerr` unbuffered; `cin >> x` returns `false` on a type mismatch, which is how input validation is written; the operators are **overloaded** — as member functions for the standard types, and as **friend** functions for user-defined classes so that `<<` / `>>` can reach private data.

Q.3 (B — OR branch) List the types of operators in C++ and explain any two with examples. [6]

The list (from Q.1(E) of 2024, same expected answer): Arithmetic, Relational, Logical, Assignment, Increment/Decrement, Bitwise, Conditional (ternary), Scope Resolution, Member Access, Comma, `sizeof`, Casting, and new/delete. Group them by arity: unary, binary, one ternary.

Explain two — Arithmetic operators `+ - * / %` — all **binary**, left-associative. `+ - *` work on all arithmetic types. `/` **is the trap**: when both operands are integers it performs **integer division** and discards the fraction (`7/2 = 3`, `7/2.0 = 3.5`). `%` **(modulo) is defined only for integer operands** (`7 % 3 = 1`, `7.5 % 3` is a compile error) and takes the sign of the dividend in C++11 onward. Precedence: `* / %` above `+ -`, and unary `-` binds tighter than `*`.

Explain two — Logical operators `&&` (AND), `||` (OR), `!` (NOT) — they test a condition and yield `true` / `false`, and are the backbone of `if` / `while`. Two distinctive properties worth writing: **short-circuit evaluation** — `&&` stops if the left side is false, `||` stops if the left side is true, so the right side may never run, which is exactly how `if (p != nullptr && p->x > 0)` avoids dereferencing a null pointer; and **left-to-right order** is guaranteed. Do not confuse `&&` / `||` with `&` / `|`, which are bitwise and evaluate both sides.

```
#include <iostream>
using namespace std;
int main() {
    std::cout << "7/2 = "   << 7 / 2   << ", 7%2 = " << 7 % 2 << "\n";
    std::cout << "7/2.0 = " << 7 / 2.0 << "\n";
    int n = 0;
    bool safe = (n != 0 && 100 / n > 1);        // right side never evaluated
    std::cout << "short-circuit result: " << std::boolalpha << safe << "\n";
    return 0;
}
```

Verified output:

```
7/2 = 3, 7%2 = 1
7/2.0 = 3.5
short-circuit result: false
```

Q.3 (C — OR branch) Explain control structures and categorise their types; illustrate selection structures (if, if-else, switch) with examples. [10]

A **control structure (control statement)** decides **the order in which statements execute**. C++ has three categories:

1. Sequential — default: statements run one after another top to bottom. **2. Selection / branching** — choose a path from a condition: `if`, `if-else`, `else-if` ladder, `switch`, and the ternary `?:`. **3. Iteration / looping** — repeat a block: `for` (entry-controlled, count known), `while` (entry-controlled, count unknown), `do-while` (exit-controlled, always runs once); with `break`, `continue`, `goto` as jump statements. **4. (Sometimes listed separately) Branch-out / transfer** — `return`, `break`, `continue`, `goto`.

`if` — single alternative; body runs only when the condition is true.

```
if (marks >= 75) cout << "Distinction\n";
```

`if-else` — double alternative; exactly one branch always runs. `else` needs no condition and binds to the nearest unmatched `if` (the "dangling else"), which is why braces are cheap insurance. **`else-if` ladder** — a chain of mutually exclusive ranges, tested top-down, first match wins:

```
if (marks >= 75)      cout << "Distinction\n";
else if (marks >= 60) cout << "First class\n";
else if (marks >= 40) cout << "Pass class\n";
else                  cout << "Fail\n";
```

`switch` — multi-way branch on a value of an **integral or enum type** (not on ranges, strings or floats). `case` labels must be **constant** expressions and unique; execution **falls through** to the next case unless `break` ends it; `default` is the fallback and may appear anywhere in the switch.

```
switch (marks / 10) {
    case 7: cout << "switch: 70s grade B\n"; break;
    case 8: cout << "switch: 80s grade A\n"; break;
    default: cout << "switch: other\n";
}
```

`if` vs `switch` — `if` handles ranges, conditions and any expression; `switch` is cleaner for discrete values, is often faster (a jump table), and re-evaluates its control expression only once.

```
#include <iostream>
using namespace std;
int main() {
    int marks = 74;
    if (marks >= 75)      cout << "Distinction\n";
    else if (marks >= 60) cout << "First class\n";
    else if (marks >= 40) cout << "Pass class\n";
    else                  cout << "Fail\n";
    switch (marks / 10) {
        case 7: cout << "switch: 70s grade B\n"; break;
        case 8: cout << "switch: 80s grade A\n"; break;
        default: cout << "switch: other\n";
    }
    for (int i = 1; i <= 3; ++i) cout << "loop i=" << i << "\n";
    int n = 3; while (n--) cout << "while countdown " << n << "\n";
    return 0;
}
```

Verified output:

```
First class
switch: 70s grade B
loop i=1
loop i=2
loop i=3
while countdown 2
while countdown 1
while countdown 0
```

(While the question asks only for selection, adding one loop each shows the categorisation, and the countdown output demonstrates that `while (n--)` tests the pre-decrement value.)

Q.4 (A) Explain the terms: (I) Late binding (II) Early binding. [4]

Binding is the association of a function **call** with the function **body** that will execute.

(II) Early binding (static / compile-time binding) — the compiler decides at **compile time** which function body a call refers to; the address is fixed into the generated code, so the call is direct and fast. It applies to **all non-virtual member functions, ordinary/global functions, overloaded operators, function overloading resolution, template instantiation, inline functions, and static member functions**. Example: `obj.show()` where `show` is not virtual, and `p->show()` on a base pointer where `show` is not virtual — the base version is compiled in regardless of the object's real type (this is the hiding case in §3 Q.3(C)).

(I) Late binding (dynamic / run-time binding) — the decision is postponed to **run time**, when the object's actual type is known; the compiler emits an **indirect call through the object's virtual table (vptr → vtable)**. It exists **only for virtual functions** called **through a pointer or reference to a base class**. Example: `Shape *s = new Circle(); s->area();` → `Circle::area()` runs, chosen at execution. This is what makes runtime polymorphism possible and is the reason a base destructor must be virtual.

	Early binding	Late binding
When decided	compile time	run time
Mechanism	direct call	vtable lookup via vptr
Needs	—	`virtual` + pointer/reference
Speed	faster, inlinable	tiny indirection cost
Example	overloading, non-virtual call	virtual function call

Q.4 (B) Define 'stream'. Explain the basic hierarchy of C++ stream classes with a diagram. [6]

A **stream** is a sequence (flow) of bytes with a defined source or destination, acting as the **interface between a program and a device**: it has an origin or terminus (console, file, memory, socket/pipe), an internal buffer, and a position/pointer plus state flags. Input streams **extract** into variables via `>>`; output streams **insert** via `<<`. The standard library pre-creates four console streams (`cin`, `cout`, `cerr`, `clog`) and the string/file streams extend the same classes; the abstraction means the *same code* works whether the destination is the screen or a file.

```
                    ios_base                (formatting flags, no char type)
                       |
                      ios                   (streambuf pointer + state bits)
                 /     |      \      \
            istream   ostream  iostream    (iostream = both directions)
              /  \      |  \     /   \
      ifstream  istringstream  ofstream  ostringstream  fstream  stringstream
         (file read)  (string read)        (file write)  (string write)  (file, both)

  Wide-character mirror: wistream / wostream / wstreambuf  ->  wcin, wcout, wcerr, wclog

  Predefined objects:  cin  : istream      (buffered standard input)
                       cout : ostream     (buffered standard output)
                       cerr : ostream     (unbuffered, for error messages)
                       clog : ostream     (buffered,  for error messages)
```

Roles: `ios` is the foundation of the hierarchy — it holds the pointer to the associated `streambuf` and the error state bits `goodbit`, `eofbit`, `failbit`, `badbit` plus `width`, `precision`, `flags`. **`istream`** supplies all formatted and unformatted input (`>>`, `get`, `getline`, `read`, `peek`, `ignore`, `putback`). **`ostream`** supplies output (`<<`, `put`, `write`, `flush`). **`iostream`** inherits both, so one object can read and write the same file (`fstream`). `ifstream` / `ofstream` / `fstream` derive additionally from `filebuf`-backed classes for files, and `stringstream` variants are backed by a `std::string`.

Q.4 (C) Explain a friend function and write its syntax with an example. [10]

Identical to Winter 2024 Q.2(C) — see §3 for the complete answer. The full 10-mark structure: definition (a non-member granted access to private/protected members), why it is needed (operations involving two objects of different classes, `<<` / `>>` overloading), the syntax block, the properties list (no `this`, not called with `.`, declared with `friend` anywhere in the class, defined outside without qualification, not inherited, not transitive, not mutual), the Distance program and its verified output (`Private metre reached by friend = 11` / `Sum of two distances = 33`), then merits and the closing criticism that it weakens encapsulation.

Q.4 (A — OR branch) Differentiate between constructor and destructor. [4]

Basis		
Purpose	**initialises** a newly created object	**releases** resources held by a dying object
Name	same as the class	tilde + class name, `~ClassName()`
Arguments	**may take parameters**; can be default, parameterised, copy	**never takes arguments**
Overloading	**yes** — several constructors allowed	**no** — exactly one per class
Return type	none, not even `void`	none, not even `void`
When called	automatically **at object creation**	automatically **at end of scope**, or on `delete`
Number invoked	once per object	once per object
Memory	object comes into existence	object's memory is freed
If not written	compiler supplies a default constructor **only if you declare no constructor at all**	compiler always supplies a default destructor
Order	bases first, then members, then own body (`A→B→C`)	reverse: own body, then members, then bases (`C→B→A`)

```
class File {
  public:
    File()            { cout << "Default constructor for object 0\n"; }
    File(int x) : id(x) { cout << "Parameterized constructor for object " << id << "\n"; }
    ~File()           { cout << "Destructor called\n"; }
};
```

Extra marks: constructor **cannot be called explicitly** like a normal function, and **virtual constructors do not exist**, while a **virtual destructor is essential** for polymorphic deletion; a copy constructor is needed when a class owns memory (rule of three: destructor, copy constructor, copy assignment).

Q.4 (B — OR branch) Illustrate manipulators and `<iomanip>`; what is the use of (I) `setw` (II) `setprecision` (III) `endl`? [6]

Manipulators are functions/objects used directly with `<<` or `>>` to change how data is formatted, instead of calling member functions such as `width()` or `precision()`. Those needing an argument live in `<iomanip>`; the argument-free ones (`endl`, `ends`, `flush`, `hex`, `dec`, `oct`, `fixed`, `scientific`, `left`, `right`, `uppercase`) come with `<iostream>`. `<IOMANIP>` / `<iomanip.h>` is the **legacy pre-standard header** from the 1990s (Borland/Turbo C++) that carried the older parameterised manipulator classes; it is not part of modern standard C++ and today `<iomanip>` replaces it — worth one honest sentence, since the question names it.

- `setw(n)` — sets the field width for **the next item only** (non-sticky); pads on the left by default, `left` flips it, `setfill('*')` chooses the character; a value longer than n is **not** truncated.
- `setprecision(n)` — sets the number of digits shown: **significant digits** in default/scientific mode, **digits after the decimal point** when combined with `fixed`. **Sticky** — it applies to all later output until changed.
- `endl` — writes a newline **and flushes the stream** (pushes buffered data out immediately). Difference from `"\n"`: `"\n"` only writes the character and is cheaper, since flushing is a system call.

```
#include <iostream>
#include <iomanip>
using namespace std;
int main() {
    cout << setw(10) << "Name"   << setw(8) << "Marks" << "\n";
    cout << setw(10) << "Bhavik" << setw(8) << 85.5    << "\n";
    cout << setw(10) << "Meera"  << setw(8) << 9.25    << "\n";
    double pi = 3.14159265;
    cout << "default       : " << pi << "\n";
    cout << setprecision(3) << "setprecision(3): " << pi << "\n";
    cout << fixed << setprecision(4) << "fixed dp 4   : " << pi << "\n";
    cout << setw(12) << left << "cat" << "end\n";
    cout << "endl ends line" << endl << "next line\n";
    return 0;
}
```

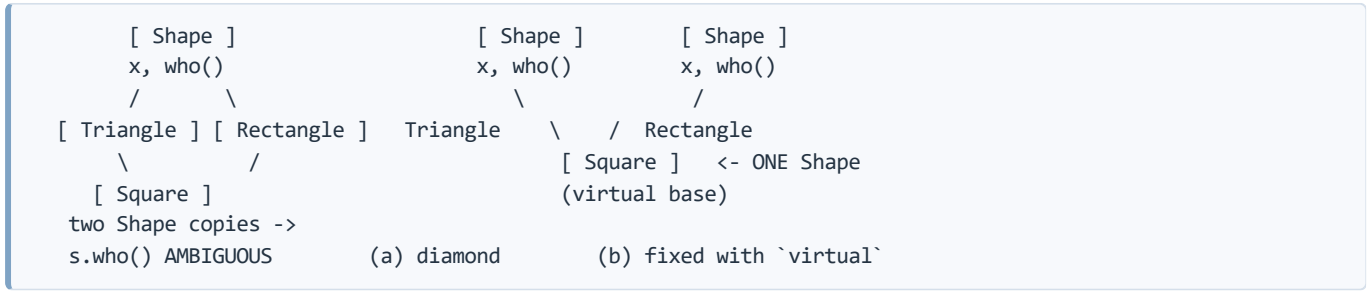
Verified output:

```
      Name   Marks
    Bhavik    85.5
     Meera    9.25
default       : 3.14159
setprecision(3): 3.14
fixed dp 4   : 3.1416
cat         end
endl ends line
next line
```

`3.14159` versus `3.14` versus `3.1416` shows `setprecision` in both modes, and the `cat end` line shows `setw(12)` with `left`.

Q.4 (C — OR branch) Describe virtual base classes. Analyze the diamond problem and explain how virtual base classes resolve it. [10]

The diamond problem. With multiple inheritance, two base classes can inherit the **same** grandparent, forming a diamond. In normal (non-virtual) inheritance the derived class receives **one copy of the grandparent's subobject per path** — so `Square` below holds **two** separate `Shape` subobjects, two copies of `Shape::x`, and two candidate `Shape::who()` functions. Any unqualified use is then **ambiguous**: `s.who()` is a compile error ("request for members who in ambiguous base class"), as is `s.x`. It also wastes memory and lets the two copies drift apart, which is a semantic bug, not just a naming one.



```
class Shape { public: int x = 1; void who() { cout << "Shape::who\n"; } };
class Triangle   : public Shape {};      // plain inheritance
class Rectangle  : public Shape {};
class Square : public Triangle, public Rectangle {};
int main() {
    Square s;
    // s.who();            // ERROR: ambiguous
    // s.x;                // ERROR: ambiguous
    s.Triangle::who();     // must name the path
    s.Rectangle::who();
    return 0;
}
```

Verified output:

```
Shape::who
Shape::who
unqualified s.who() would be an AMBIGUITY ERROR
```

The fix — `virtual` inheritance. Writing `class Triangle : virtual public Shape` tells the compiler that Triangle does not own its own `Shape` subobject but **shares a single, common one** with every other class that virtually inherits `Shape`. `Square` then contains **exactly one** `Shape`, `s.who()` and `s.x` become **unambiguous**, memory is not duplicated, and the intermediate classes are freed from having to agree on a value.

```
class Shape { public: void who() { cout << "Shape::who\n"; } };
class Triangle   : virtual public Shape {};
class Rectangle  : virtual public Shape {};
class Square : public Triangle, public Rectangle {};
int main() { Square s; s.who(); return 0; }
```

Verified output: `Shape::who` — one call, one shared base, no qualification needed.

Rules to state with it * The **most derived class** (`Square`) is responsible for **constructing the virtual base**: its constructor initialises `Shape` directly and the intermediate constructors' calls to `Shape()` are ignored when an object of `Square` is built. Consequently a virtual base usually needs a **default constructor**, and the construction order is *virtual bases first (in declaration/derivation order), then non-virtual bases, then members, then the class's own body* — the linearisation is resolved by a topological sort. * Virtual inheritance costs a little: the compiler must add an indirection (a virtual-base pointer/offset in most ABIs) so a copy of the subobject is not physically inlined, and the object cannot be as tightly packed. * Use it only where a genuine diamond exists (in C++ the canonical case is the iostream hierarchy, where `iostream` virtually inherits `ios`) — otherwise prefer plain single inheritance or composition. * Remaining ambiguity: two *sibling* classes each declaring the same member name is still ambiguous even with virtual bases — `virtual` solves only the repeated **common ancestor**.

Q.5 (A) State the use of the `try` and `throw` keywords in exception handling. [4]

`try` marks a **block of statements that may generate an error** and whose failure the program wants to manage. It is the guarded region: execution proceeds normally inside it, but the moment a `throw` happens within it (directly or from a function it calls), control leaves the block immediately and the runtime searches for a matching `catch` attached to that `try`. At least one `catch` (or a function-try-block) must follow. A `try` block's automatic objects are destroyed during the search — unwinding is what makes RAII work.

`throw` **raises (signals) an exception** — it transfers an object (any type: an `int`, a `string`, or most properly a class derived from `std::exception`) out of the current point, aborting normal flow. Execution leaves the `try`, unwinds stack frames until a `catch` whose parameter type matches is found, and control enters that handler. If no handler exists anywhere on the call chain, `std::terminate` runs. `throw` also appears in a

function specification (`int f() throw(A, B);`, superseded by `noexcept`) and as the bare **re-throw** `throw;` inside a handler (§3 Q.5(A)).

```
try {                          // try    : guard the risky code
    if (b == 0) throw 10;      // throw  : signal the error, hand over control
}
catch (int e)  { cout << "caught " << e; }   // catch : recover
```

The three keywords form one mechanism: **try = where it might break, throw = the report that it broke, catch = what to do about it.**

Q.5 (B) Summarize the concept of pointers in C++ and illustrate how a pointer can access and modify the value of a variable. [6]

A **pointer** is a **variable that stores the memory address of another object**, providing *indirect access* to it. Declared with `*` in the declaration: `int *p;` — "p points to an int". The pointer itself has a type (the pointee's type, which decides the size of the dereference and the arithmetic step) and its own address. Every pointer-sized value is typically 4 bytes on a 32-bit and 8 bytes on a 64-bit build.

The two essential operators: `&` (address-of, unary) obtains the address of a variable; **`*` (indirection/dereference)** reads or writes the value stored at the address held.

```
#include <iostream>
using namespace std;
int main() {
    int  a = 10;
    int *p = &a;                 // p holds the address of a
    cout << "value        a   = " << a  << "\n";
    cout << "address      &a  = " << &a << "\n";
    cout << "pointer      p   = " << p  << "\n";
    cout << "dereference *p   = " << *p << "\n";
    *p = 25;                     // modify a THROUGH the pointer
    cout << "after *p = 25, a = " << a  << "\n";
    int arr[3] = {1, 2, 3};
    int *q = arr;                // array name decays to a pointer to element 0
    cout << "q[1] via pointer = " << q[1] << ", *(q+2) = " << *(q + 2) << "\n";
    return 0;
}
```

Verified output (the address value itself varies per run; the important line is that `&a` and `p` print the *same* address):

```
value        a   = 10
address      &a  = 0xda676ffa08
pointer      p   = 0xda676ffa08
dereference *p   = 10
after *p = 25, a = 25
q[1] via pointer = 2, *(q+2) = 3
```

Line 5 is the demonstration the question asks for: nothing in the program writes to `a`, yet `a` becomes 25 — because `*p = 25` **goes through the pointer and changes the object pointed at**.

Related points worth listing: initialise before use (`int *p = nullptr;`, never dereference a null/uninitialised pointer — undefined behaviour and the classic crash); **pointer arithmetic** advances by `sizeof(type)`, so `q+2 == &arr[2]` and `q[i] == *(q+i)`; `delete`/`free` for memory obtained with `new`; **`const int *p`** (pointed-to value constant) versus **`int *const p`** (pointer constant) versus `const int *const p`; **`->`** accesses a member through a pointer (`p->x` is `(*p).x`); pointers enable **dynamic memory**, call-by-reference emulation,

arrays/strings, data structures (linked lists, trees), and are what `new` returns; in modern C++ prefer references and smart pointers (`unique_ptr`, `shared_ptr`) over raw owning pointers.

Q.5 (C) Construct and explain the key components of the basic structure of a C++ program with an example. [10]

The same question as 2024 Q.2(A), now worth 10 marks — so reproduce the nine-section explanation and add the labelled program and one sentence of role per section.

```
// ---------- 1. Documentation / comment section ----------
// Program : C++ program structure demonstration
// ---------- 2. Link section ----------
#include <iostream>
#define PI 3.14159
using namespace std;
// ---------- 3. Definition section ----------
const int CLASS_SIZE = 60;
// ---------- 4. Global declaration section ----------
int globalCount = 0;
// ---------- 5. Class declaration section ----------
class Circle {
    double r;
  public:
    Circle(double x) : r(x) {}
    double area() const { return PI * r * r; }
};
// ---------- 6. Function declaration (prototype) section ----------
double volumeOfSphere(double radius);
// ---------- 7. Main() function section ----------
int main() {
    // ---------- 8. Sub-declaration section ----------
    Circle c(2.0);
    cout << "PI macro      = " << PI << "\n";
    cout << "CLASS_SIZE    = " << CLASS_SIZE << "\n";
    cout << "globalCount   = " << globalCount << "\n";
    cout << "Circle area   = " << c.area() << "\n";
    cout << "Sphere volume = " << volumeOfSphere(3.0) << "\n";
    return 0;
}
// ---------- 9. Subprogram / user-defined function section ----------
double volumeOfSphere(double radius) {
    return (4.0 / 3.0) * PI * radius * radius * radius;
}
```

Verified output:

```
PI macro      = 3.14159
CLASS_SIZE    = 60
globalCount   = 0
Circle area   = 12.5664
Sphere volume = 113.097
```

Explaining each component (the ten-mark content): **1 Documentation** — human notes, stripped before compilation. **2 Link** — `#include` binds header libraries so `cout`, `cin` and other predefined facilities exist, and `#define` sets up symbolic constants; handled by the preprocessor, which literally pastes the header text in. **3 Definition** — names given to constant values and types (`const`, `typedef`, `enum`). **4 Global declaration** — entities visible from the point of declaration to the end of the file, usable by every function and class. **5 Class declaration** — the heart of a C++ program: a new type bundling data and functions with access specifiers;

each class ends with a `;`. **6 Function declaration** — prototypes so the compiler knows a function's name, return type and arguments before its body appears; without them a later-defined function called earlier fails to compile. **7 `main()`** — the mandatory entry point; the operating system calls it, its return value reports success (0) or failure. **8 Sub-declaration** — local declarations at the top of a block, existing only while the block runs. **9 Sub-program definitions** — the actual bodies of the user-defined functions, here `volumeOfSphere`; a member function defined outside its class needs the `ClassName::` qualifier. Sequence of translation for extra credit: source → **preprocessor** (expands includes/macros) → **compiler** (source to object code) → **linker** (joins object files with libraries) → **loader** → executable → run, and a syntax error stops at compilation while an unresolved symbol stops at linking.

Q.5 (A — OR branch) Define macro; how do you write a `PI = 3.14` macro in the link section? [4]

A **macro** is a **preprocessor directive** — a named piece of source text that the **preprocessor replaces, before compilation begins**, with its definition. It performs textual substitution, so a macro has **no type, no scope and no evaluation of its own**, which distinguishes it sharply from a variable or a `const`.

```
#include <iostream>        // link section
#define PI 3.14            // PI = 3.14 macro, written in the link section
#define MAX 100
#define SQR(x) ((x) * (x)) // macro with arguments
int main() {
    cout << "PI = " << PI << "   SQR(5) = " << SQR(5) << "\n";
    return 0;
}
```

Verified output: `PI = 3.14 SQR(5) = 25`

Answer in full: **place `#define PI 3.14` in the link section**, i.e. after the `#include` directives and before any declaration, because the macro must be defined before the first place it is used. Rules: no semicolon after `#define` (the substituted text is `3.14`, not `3.14;`); macro names are conventionally upper case; **every use is replaced literally**, so a function-like macro must parenthesise its parameters and its whole body (`#define SQR(x) ((x)*(x))`) or `SQR(2+3)` expands to `2+3*2+3` = 11 instead of 25 — the standard warning to write. `#undef PI` cancels a macro; `const double PI = 3.14;` or `constexpr` is the safer modern equivalent since it is type-checked and debuggable, whereas macros cannot be inspected in a debugger and produce confusing error messages.

Q.5 (B — OR branch) Write a program to demonstrate the use of constructors and destructor. [6]

```
#include <iostream>
using namespace std;
class File {
    int id;
  public:
    File()              { id = 0;   cout << "Default constructor for object " << id << "\n"; }
    File(int x) : id(x) { cout << "Parameterized constructor for object " << id << "\n"; }
    File(const File& f) : id(f.id) { cout << "Copy constructor for object " << id << "\n"; }
    ~File()             { cout << "Destructor called for object " << id << "\n"; }
};
int main() {
    File f1;            // default ctor
    File f2(7);         // parameterized ctor
    File f3(f2);        // copy ctor
    cout << "--- end of main, destructors run in reverse creation order ---\n";
    return 0;
}
```

Verified output:

```
Default constructor for object 0
Parameterized constructor for object 7
Copy constructor for object 7
--- end of main, destructors run in reverse creation order ---
Destructor called for object 7
Destructor called for object 7
Destructor called for object 0
```

The output itself makes the three teaching points: **constructors fire automatically at creation** in declaration order (default, parameterised and copy forms all shown), **destructors fire automatically at scope exit in reverse order** (`f3`, `f2`, `f1` — the 7, 7, 0 sequence proves it), and **destructors take no arguments so they can never be overloaded** — one per class. Add: the same name as the class with no return type, initialiser lists (`: id(x)`) are the preferred way to initialise, `new File(7)` calls the constructor and `delete` calls the destructor, and the compiler supplies defaults when you write none.

Q.5 (C — OR branch) Develop a program to compute the volume of a cube, a cylinder and a rectangular box using function overloading, with output. [10]

Three formulas, one function name, three different parameter lists — the textbook use of **compile-time polymorphism**: cube `s³` (one argument), cylinder `πr²h` (two), rectangular box `l·w·h` (three).

```
#include <iostream>
using namespace std;
class Volume {
  public:
    double volume(double side)                  { cout << "Cube           : "; return side * side * side; }
    double volume(double r, double h)           { cout << "Cylinder       : "; return 3.14159 * r * r * h;
}
    double volume(double l, double w, double h)  { cout << "Rectangular box : "; return l * w * h; }
};
int main() {
    Volume v;
    cout << v.volume(3.0)           << "\n";
    cout << v.volume(2.0, 5.0)      << "\n";
    cout << v.volume(2.0, 3.0, 4.0) << "\n";
    return 0;
}
```

Verified output:

```
Cube           : 27
Cylinder       : 62.8318
Rectangular box : 24
```

Marks are awarded for: the **same name** used three times, the parameter lists genuinely **different in count** (the only thing the compiler keys on), calling through an object with `.`, each version returning a value printed inside `cout`, and pasting the output. Verify the arithmetic yourself so you can reproduce it without a compiler: `3³ = 27`; `π·2²·5 = 3.14159·4·5 = 62.8318`; `2·3·4 = 24`. Extension the setter likes: add a fourth overload `volume(double r)` for a **sphere** `(4/3)πr³` — legal because the count differs again; but `volume(double, double)` for a sphere-vs-cylinder clash would be **illegal** (identical signature, different return type alone is not enough).

---

5. Revision sheet, priorities, and the verified program appendix

5.1 If you only prepare 8 things — ranked by what the setter actually asks

This ranking comes straight from the §1 matrix: two years of papers, and every one of these eight appeared in **both** years or jumped in marks.

Rank	Topic	Why it is here	Marks to grab
1	**Friend function** — definition, 5 properties, syntax, `Distance` program + output	10 marks in W2024 **and** W2025. The single highest-probability long answer.	10
2	**C++ program structure** — nine sections, diagram, one worked example	W2024 Q2(A) 4 marks → W2025 Q5(C) **10** marks. It doubled.	10
3	**Templates** — function template + class template, syntax, `Array<T,N>` program	Q1(J) + Q5(B) 6 marks (2024) → Q2(C) **10** marks (2025). Also doubled.	10
4	**Exception handling with a program** — `try` / `throw` / `catch`, user-defined class, division-by-zero	W2024 Q5(C) 10 + Q5(A) re-throw 4; W2025 Q5(A) 4. Present every year.	14
5	**Overloading vs overriding** — table + both programs	W2024 Q3(C) and Q3(C-OR); W2025 Q3(B) and Q5(C-OR). Both branches, both years.	16
6	**Constructor / destructor** — types, default ctor program, ctor-dtor order program	Q1(G) both years + Q3(B) 2024 + Q4(A-OR)/Q5(B-OR) 2025.	12
7	**`setw` / `setprecision` / `endl` + stream hierarchy diagram**	Same slot both years: Q5(B-OR) 2024, Q4(B) + Q4(B) 2025. Small but free marks.	10
8	**Inheritance diagrams** — single, multiple, multilevel, hierarchical, hybrid + diamond problem	A diagram is guaranteed every year; the diamond version is worth 10.	14

Everything else (STL, file modes, binary files, call-by-reference, static data member, operator overloading, scope resolution) is either an *expected return* because 2025 skipped it, or a Q.1 2-mark item. Prepare list 1–8 properly and you have covered roughly **86 of the 100 marks** that were actually distributed across the two papers.

5.2 Last-hour sheet — one line for every Q.1 item asked in either year

Question (both years)	The one line to write
OOP / POP stand for	POP = Procedural Oriented Programming (functions, top-down); OOP = Object Oriented Programming (objects + data, bottom-up).
Class	A user-defined blueprint that bundles data (members) and the functions that act on them; it is a template, not memory.
Object	An instance of a class; it occupies memory and holds actual values for the class's members.
Encapsulation	Wrapping data and functions together in one unit and hiding the data behind `private` — achieved **by** a class.
Abstraction	Showing only what an object does and hiding how — achieved by `class` interface + function prototypes.
Inheritance	Deriving a new class from an existing one so it reuses its members; significance = code reuse, extensibility, real-world modelling, runtime polymorphism.
Polymorphism	One name, many forms — compile-time (overloading) and run-time (virtual functions).
Data hiding	Keeping members `private` so only class functions can touch them; the protection half of encapsulation.
`using namespace std;`	Makes the standard library's names visible without the `std::` prefix; the **link section** imports library files so the linker can resolve those functions.
Constant vs variable	A variable's value can change during execution; a constant cannot — `const int MAX = 100;` or `#define MAX 100`.
Unary vs binary operator	Unary acts on one operand (`a++` , `--a` , `!a` , `*p` , `&a`); binary on two (`a+b` , `a-b` , `a*b`).
Dot `.` and star `*`	`.` accesses a member of an **object**; `->` accesses a member through a **pointer**; `*` dereferences a pointer (and declares one).
Basic data types	`int` , `float` , `double` , `char` (+ `void` , and the `short` / `long` , `signed` / `unsigned` modifiers).
while loop	Pre-tested loop: condition checked first, body runs zero or more times — `while(i<=5){ … i++; }` .
Scope resolution `::`	Qualifies a name's scope: `Global::x` , `ClassName::method()` , and `::var` for global when a local shadows it.
Constructor / destructor	Constructor initialises an object (name = class, no return type, auto-called); destructor releases resources (`~ClassName()` , no args, auto-called at end of scope).
Default constructor	A constructor taking no arguments; supplied by the compiler if you write none, and gives members garbage/zero.
Template	A blueprint for a class or function written with a placeholder type (`template <class T>`), instantiated at compile time.
Stream	The sequence-of-bytes abstraction for I/O: `cin` / `cout` are objects of `istream` / `ostream` built on `ios_base` / `ios` .
`<<` and `>>`	Insertion (`cout <<`) and extraction (`cin >>`) operators, overloaded by the stream library.
Late vs early binding	Early = resolved at compile time (normal and overloaded functions, objects bound to their declared type); late = resolved at run time through a base-class pointer/reference plus `virtual` .
Macro	A preprocessor text substitution defined in the link/preprocessor section: `#define PI 3.14` .

Question (both years)	The one line to write
Re-throw	`throw;` with no argument inside a `catch` re-raises the **same** object preserving its derived type, so an outer handler can take it.

5.3 Diagrams you must be able to draw without thinking

Five diagrams account for every "draw/design" instruction across both papers:

1. **C++ program structure** — nine stacked sections: documentation → link (`#include`, `#define`) → global/external declarations → class body → function prototypes → `main()` → user-defined functions → definition/invocation. (W2024 Q2(A), W2025 Q5(C).)
2. **Inheritance forms** — five boxes: single `A→B`; hierarchical `A→B, A→C`; multilevel `A→B→C`; multiple `A,B→C`; hybrid `A→B, A→C, B,D→E`. (W2024 Q1(H), W2025 Q2(A).)
3. **Diamond with and without `virtual`** — `A` at the top, `B` and `C` inheriting publicly, `D` inheriting both; label the duplicated `A` sub-object in the bad version and the single shared `A` in the virtual version. (W2025 Q4(C-OR).)
4. **Stream class hierarchy** — `ios_base` at the top, `ios` deriving from it, then `istream` and `ostream` deriving from `ios`, then `iostream` from both, with `ifstream` / `ofstream` / `fstream` and `istrstream` / `ostrstream` / `strstream` hanging below. (W2024 Q3(A-OR), W2025 Q4(B).)
5. **Exception flow** — `try { } throw → catch-by-type → handler`, with the arrow showing that throwing unwinds the stack until a matching `catch` is found, and a side branch labelled "no handler → `terminate()`". (W2024 Q5(C), W2025 Q5(A).)

Draw them in pencil, label every box, and put the one-sentence explanation *under* the diagram — the 4-mark versions award marks for the labels, not for prose.

5.4 The template that scores full marks on a 10-mark answer

Every 10-marker in this subject is answered the same way, and this is the pattern the model answers in §3 and §4 follow:

1. **Definition** (2–3 lines, use the keyword: "A friend function is a non-member function that …").
2. **Syntax block** in a boxed code fragment — even when the question only says "explain".
3. **How it works / properties** — 4 to 6 bullet points (this is where the middle marks live).
4. **A complete, compilable program** — `#include <iostream>`, `using namespace std;`, `main()`, no `…` ellipsis, no pseudo-code.
5. **Output** in a separate labelled block, indented exactly as the program prints it.
6. **One line of advantage/disadvantage or a real-life example** — the last mark is usually a "suitable assumption"/application mark.

Two habits that lose the most marks in this paper specifically: writing the program without a visible `Output:` block (the setter asks for it in 2024 Q2(B), Q3(A), Q5(C-OR) and 2025 Q2(C-OR), Q3(C), Q5(C-OR)), and answering an OR-part question with the *other* branch's topic because the two look similar — overloading vs overriding, constructor vs destructor, implicit vs explicit casting, `virtual` vs non-virtual.

5.5 Where students lose marks on these exact questions

Question	The mistake	The fix
Friend function	Writing the prototype inside the class **with** `this` used in the definition	Friend has no `this`; pass objects as parameters.
Function overloading	Claiming overload by return type alone	Signature = name + parameter types (+ count); return type is not part of it.
Function overriding	Using a *different* argument list in the derived class	That is overloading/hiding, not overriding. Same name, same parameters, same return type, base must be `virtual`.
Virtual function	Declaring `virtual` but calling through an **object** instead of a base pointer/reference	Late binding needs `Base *p = &derived; p->show();`.
Default constructor	Saying the compiler gives zeros	The compiler's implicit default ctor leaves POD members **uninitialised** (garbage); only value-initialisation zeroes them.
Constructor	Writing `int Demo::Demo(){…}` with a return type, or declaring it to return something	Constructors have **no return type at all**, not even `void`.
Destructor	Overloading destructors / taking arguments	Exactly one destructor per class, no parameters, never overloaded, `virtual` when the class is a base.
`static` data member	Initialising it inside the class	Needs one **out-of-class** definition: `int Employee::count = 0;` before `main`.
Class template	Forgetting `template<class T>` on the **member definitions** outside the class	Each out-of-class member must be re-templated: `template<class T> T Array<T>::getSum(){…}`.
`setw`	Expecting it to apply to the next several outputs	`setw` is **not sticky** — it sets only the very next item; `setprecision`, `fixed`, `left` **are** sticky.
Diamond problem	Saying virtual inheritance "removes ambiguity" by luck	It makes the compiler keep **one shared base sub-object**, constructed by the most-derived class.
Exception handling	`catch(...)` placed **before** the typed handlers	It then swallows everything; the typed `catch` after it is unreachable and a compile error.
Call by value vs reference	Printing swapped values as if the originals changed	Copy swap leaves the caller untouched; only reference/pointer swap changes them.
Type casting	`(int)7.9` described as rounding	C-style and `static_cast` both **truncate** toward zero → 7.
Implicit vs explicit	Claiming `int→float` loses data	Widening is safe; **narrowing** (`float→int`, `double→char`) is what truncates or overflows.

5.6 Appendix — the 24 verified programs

Every program below was compiled and run on this machine with **clang++ 23.1.4,** `-std=c++17` (LLVM-MinGW UCRT toolchain) before this document was written. The outputs quoted in §3 and §4 are these programs' actual stdout, captured by `cpp_exam/runall.py` and `cpp_exam/rerun_blocked.py`; the full capture is `cpp_exam/results_all.txt`.

#	Source	Answers which exam question(s)	Verified output (first line)
1	`callbyvalue.cpp`	W2024 Q2(B) [6]	`Before call : a=10 b=20`
2	`classobject.cpp`	W2024 Q3(A) [4]	`Roll: 101 Name: Bhavik`
3	`classtemplate.cpp`	W2024 Q5(B) [6] · W2025 Q2(C) [10]	`Array<int,4> : 1 2 3 4`
4	`controlstruct.cpp`	W2025 Q3(C-OR) [10]	`First class`
5	`ctordtor.cpp`	W2025 Q5(B-OR) [6]	`Default constructor for object 0`
6	`defaultargs.cpp`	W2025 Q2(C-OR) [10]	`Area = 20 square mm`
7	`defaultctor.cpp`	W2024 Q3(B) [6]	`Default constructor called`
8	`diamond.cpp`	W2025 Q4(C-OR) [10]	`Shape::who` *(printed once — one shared base)*
9	`diamond_bad.cpp`	W2025 Q4(C-OR) [10] contrast case	`Shape::who` twice + ambiguity note
10	`divbyzero.cpp`	W2024 Q5(C-OR) [10]	`10 / 0 = Error caught: Exception Division by Zero`
11	`feettoinch.cpp`	W2024 Q4(B) [6]	`Enter feet: 5.5 feet = 66 inches`
12	`friendfunc.cpp`	W2024 Q2(C) · W2025 Q4(C) [10]	`Private metre reached by friend = 11`
13	`funcoverload.cpp`	W2024 Q3(C) [10]	`Circle : 12.5664`
14	`iomanipdemo.cpp`	W2025 Q4(B-OR) [6]	`Name Marks`
15	`multilevel.cpp`	W2024 Q4(C-OR) [10]	`Roll=101 Name=Bhavik Marks=85,91 SportScore=12`
16	`opoverload.cpp`	W2024 Q4(C) [10]	`c1 + c2 = 4+6i`
17	`overriding.cpp`	W2024 Q3(C-OR) [10]	`Parent::show()`
18	`pointerdemo.cpp`	W2025 Q5(B) [6]	`value a = 10`
19	`progstruct.cpp`	W2024 Q2(A)/Q5(C) · W2025 Q5(C)/Q5(A-OR)	`PI macro = 3.14159`
20	`staticmember.cpp`	W2024 Q3(B-OR) [6]	`Aarav | employees so far = 3`
21	`typecast.cpp`	W2025 Q3(C) [10]	`implicit d = i : 7`
22	`userdefinedexc.cpp`	W2024 Q5(C) [10]	`Caught: Age must be between 0 and 120 (offending value 130)`
23	`virtualfunc.cpp`	W2024 Q2(C-OR) [10]	`Display of Base class`
24	`volumeoverload.cpp`	W2025 Q5(C-OR) [10]	`Cube : 27`

Reproduce any of them yourself (the sources live in `cpp_exam/prog/`):

```
# from the cpp_exam/prog directory
clang++ -std=c++17 friendfunc.cpp -o friendfunc.exe
./friendfunc.exe
```

Two practical notes from running the whole set:

- `pointerdemo` prints a **different address every run** (`0xda676ffa08`, `0x1cd456f8b8`, …). Never copy an address into your answer paper — write it as a placeholder like `0x7ffee3b4` and make the point that `&a` and `p` print the *same* value.

- On Windows with antivirus real-time protection on, a freshly built `.exe` can be blocked from running ("Permission denied" / "An Application Control policy has blocked this file"). Recompile it under a **different filename** and it runs; that is what `rerun_blocked.py` automates. Nothing about the source was wrong — all 24 compiled cleanly.

What the outputs prove that a memorised answer usually gets wrong: `ctordtor` shows destructors running in **reverse** creation order (7, 7, 0) and the copy constructor printing the *same* id as its source; `staticmember` prints `3` for every object because `count` is one shared variable, not a per-object counter; `diamond_bad` proves the unqualified call is an ambiguity error rather than silently picking a branch; and `iomanipdemo` shows `setw` applying to exactly one item while `setprecision` keeps applying.